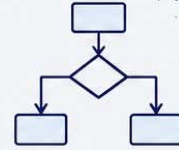


1010101010
0101010101
1010101010
0101010101



Infographics



Visual Learning

Better understanding through visual content.



Simplified Concepts

Complex topics explained in simple and visual form.



Quick Revision

Easy to revise important points and diagrams.

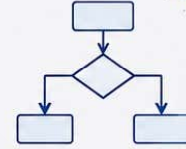


Exam Ready

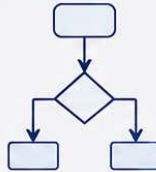
Helps in faster revision and better exam preparation.



1010101010
0101010101
1010101010
0101010101



10110
01001
10101





Introduction to Data Structure



1. What is Data Structure?

- A data structure is a way of **organizing, storing and managing data** in a computer so that it can be accessed and modified **efficiently**.
- It defines the relationship between data and the operations that can be performed on that data.
- A good data structure helps to improve the **performance** and **manage** large amounts of data easily.

2. Need of Data Structure

- Efficient data storage and retrieval.
- Reduce time complexity of operations.
- Handle large and complex data.
- Helps in solving real-world problems (e.g., databases, operating systems, computer networks, etc.).
- Improve program performance.
- Code becomes more organized and maintainable.
- Used in many areas like AI, ML, Web development, etc.



3. Applications of Data Structure

- Database Management Systems** (Indexing, searching, fast retrieval)
- Operating Systems** (Process scheduling, memory management)
- Computer Networks** (Routing, data transfer)
- Searching and Sorting** (Searching data in large collections)
- Compiler Design** (Syntax tree, symbol table)
- Artificial Intelligence & Machine Learning** (Handling large datasets)
- Web Development** (Stacks, queues, trees, graphs)

4. Characteristics of a Good Data Structure

- Efficiency** – operations should be fast (time and space efficient).
- Simplicity** – easy to understand and implement.
- Flexibility** – should allow easy modification.
- Reusability** – can be used in different applications.
- Memory utilization** – should use memory efficiently.
- Maintainability** – code should be easy to maintain.



5. Types of Data Structure

Data Structures

Primitive Data Structure (Built-in types)

- int
- float
- char
- double
- boolean

These are basic data types provided by the programming language.

Non-Primitive Data Structure (Derived types)

Linear Data Structure (Elements are arranged in a sequence)

- Array
- Linked List
- Stack
- Queue

Non-linear Data Structure (Elements are not in sequence)

- Tree
- Graph

6. Primitive vs Non-Primitive

Primitive Data Structure

- Basic data types provided by the language.
- Store single value.
- Fixed size.
- Directly supported by hardware.
- Example: int, float, char

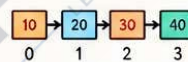
Non-Primitive Data Structure

- Derived data types built using primitive types.
- Can store multiple values.
- Variable size.
- Implemented using algorithms.
- Example: Array, Linked List, Stack, Queue, Tree, Graph

7. Linear vs Non-Linear

Linear Data Structure

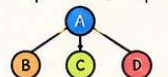
- Elements are arranged in a sequential order.
- Each element has a unique predecessor and successor (except first and last).
- Traversed in a single run.
- Examples: Array, Linked List, Stack, Queue.



Linear (Array Example)

Non-Linear Data Structure

- Elements are not arranged in a sequence.
- A single element can be connected to multiple elements.
- Used to represent hierarchical or network relationships.
- Examples: Tree, Graph.



Non-Linear (Tree Example)

8. Common Operations on Data Structure

- Traversing** – Accessing each element once.
- Insertion** – Adding a new element.
- Deletion** – Removing an existing element.
- Searching** – Finding a specific element.
- Sorting** – Arranging elements in a particular order.
- Merging** – Combining two or more data structures.

9. Real-World Examples

Array



Storing marks of students.

Stack



Undo operation in text editor.

Queue



People standing in a queue.

Tree



Folder structure in computer.

10. Summary

- Data structure is a way of organizing and storing data efficiently.
- There are two types: Primitive and Non-primitive.
- Non-primitive is further divided into Linear and Non-linear.
- Common operations include traversing, insertion, deletion, searching and sorting.
- Data structures are widely used in real-world applications to solve complex problems efficiently.





Data Structure helps us to organize, store and manage data efficiently.

Types of Data Structure

Primitive and Non-Primitive

Types of Data Structure

1. Primitive (Built-in)
2. Non-Primitive (Derived)



1. Primitive Data Structure (Built-in)

Definition

Primitive data structures are the **basic data types** provided by the programming language. They are used to store **simple, single values**.

Characteristics

- Inbuilt in the language.
- Store single value at a time.
- Fixed size.
- Directly supported by hardware.
- Faster to access and operate.
- Used to store simple data.

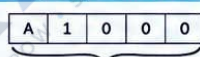
Types of Primitive Data Structure

Data Type	Description	Example	Memory Size (in C)
int	Stores integer (whole number)	10, -5, 100	2 or 4 bytes
float	Stores decimal numbers	3.14, -0.5	4 bytes
double	Stores large decimal numbers	3.14159	8 bytes
char	Stores a single character	'A', 'z', '5'	1 byte
boolean	Stores true or false value	true, false	1 byte

Example (C Language)

```
int a = 10;           // integer value
float b = 3.14;        // floating value
double c = 9.876;      // double value
char ch = 'A';         // character value
bool flag = true;      // boolean value
```

Memory Representation



Each data type occupies fixed memory location according to its size.

2. Non-Primitive Data Structure (Derived)

Definition

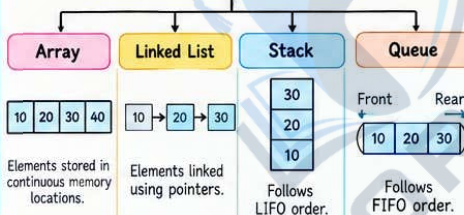
Non-primitive data structures are **derived data types** built using primitive data types. They can store **multiple values** and represent complex data relationships.

Characteristics

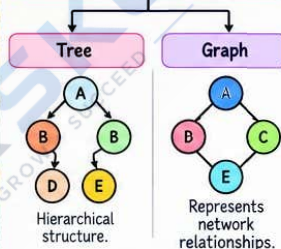
- Not inbuilt, created using primitive types.
- Can store multiple values.
- Variable size (dynamic in many cases).
- Used to represent complex data.
- Can be linear or non-linear.
- Provide efficient data organization.
- Used in real-world applications.

Types of Non-Primitive Data Structure

Linear Data Structure (Elements are arranged in a sequence)



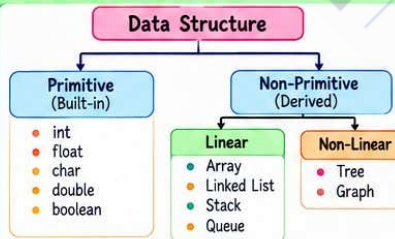
Non-Linear Data Structure (Elements are not arranged in a sequence)



3. Comparison: Primitive vs Non-Primitive

Feature	Primitive Data Structure	Non-Primitive Data Structure
Definition	Basic data types provided by language	Derived data types built using primitive types
Data Stored	Single value	Multiple values
Size	Fixed size	Variable size
Implementation	Inbuilt in language	Created by programmer
Memory	Directly supported by hardware	Uses memory allocation implementation
Complexity	Simple	Complex
Examples	int, float, char, double, boolean	Array, Linked List, Stack, Queue, Tree, Graph

4. Classification Tree



5. Key Takeaways

- Primitive data structures store simple, single values.
- Non-primitive data structures store multiple values and complex relationships.
- Primitive types are inbuilt and fixed in size.
- Non-primitive types are created using primitive types.
- Non-primitive are further classified into Linear and Non-linear.
- Both types are important for efficient data organization and problem solving.



Linear Data Structures store elements in a sequential manner, one after another.

Linear Data Structure

Examples:
1. Array
2. Linked List
3. Stack
4. Queue



1. What is Linear Data Structure?

- A linear data structure is a collection of data elements arranged in a **sequential order**, where each element has a unique predecessor and successor (except the first and last element).
- Elements are stored in a **single level** (i.e., one after another).
- Traversal is done in a **linear manner** (from start to end).
- Examples:** Array, Linked List, Stack, Queue.

2. Characteristics

- Elements are stored in a **sequential order**.
- Each element (except first and last) has a **unique predecessor and successor**.
- Can be implemented using **arrays or pointers**.
- Easy to traverse and implement.
- Require **linear time** to access elements (in most cases).
- Memory can be contiguous (Array) or non-contiguous (Linked List).
- Examples:** Array, Linked List, Stack, Queue.

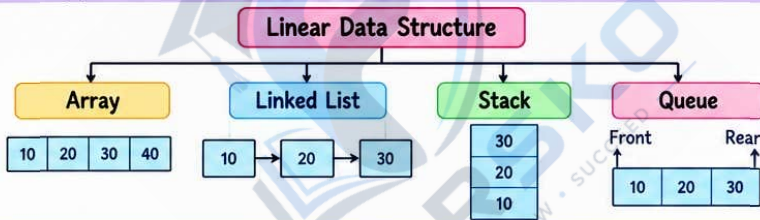
3. Advantages

- Simple to understand and implement.
- Easy to traverse.
- Efficient for small to medium sized data.
- Widely used in real-world applications.
- Some structures (Stack/Queue) are useful in specific problem-solving scenarios (e.g., recursion, scheduling).

4. Disadvantages

- Linear search is time-consuming ($O(n)$).
- Insertion and deletion can be costly (in some cases).
- Array has fixed size (in static implementation).
- May not be efficient for large and complex data compared to non-linear data structures like trees or graphs.

5. Types of Linear Data Structure



7. Array (Linear Data Structure)

- An array is a collection of similar data items stored in contiguous memory locations.
- Each element is accessed using an index (starting from 0).
- Array has a fixed size.
- Supports fast access but insertion and deletion can be costly.

Representation:

Index: 0 1 2 3 4
Array: 10 20 30 40 50

Operations:

- Traversing:** Access each element one by one.
- Insertion:** Add element at a specific position.
- Deletion:** Remove element from a specific position.
- Searching:** Find a specific element.

8. Linked List (Linear Data Structure)

- A linked list is a collection of nodes where each node contains data and a reference (pointer) to the next node.
- Elements are stored in non-contiguous memory locations.
- Dynamic in size (can grow or shrink at runtime).
- Types:** Singly Linked List, Doubly Linked List, Circular Linked List.

Representation (Singly Linked List):



Operations:

- Traversing:** Visit each node sequentially.
- Insertion:** At beginning, end or specific position.
- Deletion:** Remove a node from given position.
- Searching:** Find a node with specific value.

6. Comparison of Linear Data Structures

Structure	Storage	Access	Insertion Deletion	Structure	Key Feature
Array	Contiguous memory	Index-based ($O(1)$)	Costly ($O(n)$)	Sequential elements	Fixed size, fast access
Linked List	Non-contiguous memory	Sequential ($O(n)$)	Easy ($O(1)$)	Nodes with pointers	Dynamic size.
Stack	Contiguous/ Non-contiguous	Top element ($O(1)$)	Easy ($O(1)$)	LIFO	Used in function calls, recursion
Queue	Contiguous/ Non-contiguous	Front/Rear ($O(1)$)	Easy ($O(1)$)	FIFO	Used in scheduling, BFS

9. Stack (Linear Data Structure)

- A stack is a linear data structure that follows LIFO (Last In First Out) order.
- Insertion is called **Push operation**.
- Deletion is called **Pop operation**.
- We can insert and delete elements only from the top.

Representation:



Operations:

- Push():** Insert an element.
- Pop():** Delete an element.
- Peek()/Top():** View top element.
- isEmpty():** Check if stack is empty.
- isFull():** Check if stack is full (in array).

10. Queue (Linear Data Structure)

- A queue is a linear data structure that follows FIFO (First In First Out) order.
- Insertion is done at Rear.
- Deletion is done from Front.
- We can insert at rear and delete from front.
- Types:** Simple Queue, Circular Queue, Priority Queue, Double Ended Queue (Deque).

Representation (Simple Queue):



Operations:

- Enqueue():** Insert an element at rear.
- Dequeue():** Delete an element from front.
- Front():** View front element.
- Rear():** View rear element.
- isEmpty():** Check if queue is empty.
- isFull():** Check if queue is full (in array implementation).



Non-linear data structures do not store elements in a sequential order, and elements can be connected in a hierarchical or network manner.

Non-Linear Data Structure

Examples:
1. Tree
2. Graph



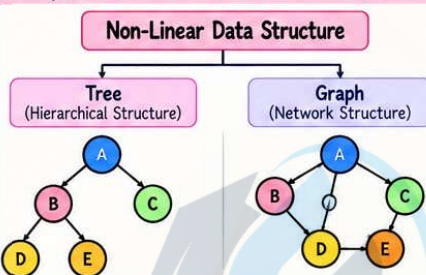
1. What is Non-Linear Data Structure?

- A non-linear data structure is a collection of data elements that are **not arranged in a sequential order**.
- Elements can be connected in a **hierarchical** or **network manner**.
- There is no single way to traverse all elements.
- Used to represent complex relationships between data.
- Examples:** Tree, Graph.

2. Characteristics

- Elements are not stored sequentially.
- Data is organized in hierarchical or network form.
- Each element can have multiple connections.
- Used to represent real-world problems (e.g., file systems, social networks).
- Supports dynamic growth.
- More complex implementation than linear data structures.

3. Types of Non-Linear Data Structure



4. Applications

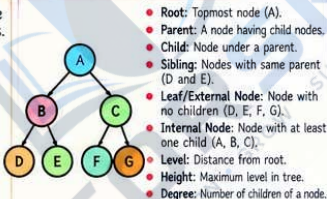
- File system directory structure (Tree).
- Expression tree in compilers (Tree).
- Database indexing (B-Tree).
- Social networks (Graph).
- Web pages linking (Graph).
- Road networks (Graph).
- AI and decision making (Trees and Graphs).
- Network routing (Graph).

5. Tree (Non-Linear Data Structure)

5.1 What is a Tree?

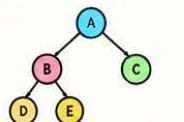
- A tree is a hierarchical data structure consisting of nodes connected by edges.
- It has a single root node and zero or more child nodes.
- There is exactly one path between any two nodes.

5.2 Terminologies in Tree



5.4 Tree Traversals

- Preorder** (Root → Left → Right)
- Inorder** (Left → Root → Right)
- Postorder** (Left → Right → Root)



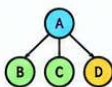
Preorder:	A	B	D	E	C	F
Inorder:	D	B	E	A	C	F
Postorder:	D	E	B	F	C	A

5.5 Operations on Tree

- Creation of tree
- Traversal (Preorder, Inorder, Postorder)
- Insertion (in BST)
- Deletion (in BST)
- Searching
- Finding height, leaf nodes, internal nodes
- Finding minimum and maximum element (in BST)

5.3 Types of Trees

General Tree



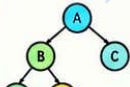
Each node can have any number of children.

Binary Tree



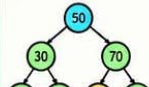
Each node has at most 2 children.

Complete Binary Tree



All levels are completely filled except possibly the last level, which is filled from left to right.

Binary Search Tree (BST)



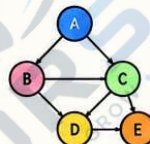
Left subtree < root < right subtree.

6. Graph (Non-Linear Data Structure)

6.1 What is a Graph?

- A graph is a non-linear data structure consisting of a set of vertices (nodes) and a set of edges that connect pairs of vertices.
- Used to represent network relationships between objects.
- Graphs can be used to model real-world problems such as social networks, transportation networks, etc.

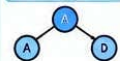
6.2 Terminologies in Graph



- Vertex (Node):** A, B, C, D, E
- Edge:** Connection between two vertices (e.g., A-B)
- Degree:** Number of edges connected to a vertex (e.g., deg(A) = 2)
- Adjacent Vertices:** Vertices connected by an edge (e.g., A and B are adjacent)
- Path:** Sequence of vertices connected by edges (e.g., A → B → D)
- Weight:** Value assigned to an edge (in weighted graph).

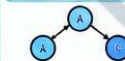
6.3 Types of Graphs

Undirected Graph



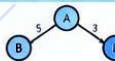
Edges have no direction.

Directed Graph (Digraph)



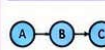
Edges have direction (from one vertex to another).

Weighted Graph



Edges have weights (values).

Connected Graph



There is a path between every pair of vertices.

Disconnected Graph



Not all vertices are connected.

Cyclic Graph



Contains a cycle (start and end at same vertex).

Acyclic Graph



Does not contain any cycle.

6.4 Representation of Graph

Adjacency Matrix

	A	B	C
A	0	1	1
B	1	0	0
C	1	0	0

A 2D array of size $n \times n$ (where n = number of vertices).

Adjacency List

A	→ B → C
B	→ A
C	→ A

Each vertex stores a list of its adjacent vertices.



Operations help us to access, modify, and manage the data stored in a data structure.



Data Structure Operations

Common operations that can be performed on data structures

Key Operations:

- Traversing
- Insertion
- Deletion
- Searching
- Sorting
- Merging



1. What are Data Structure Operations?

- Operations are the basic actions that can be performed on the data stored in a data structure.
- They help us to **access, modify, add, remove, search, arrange and manage** the data efficiently.
- Different data structures support different operations depending on their structure.
- These operations are fundamental for solving real-world problems and implementing algorithms.

2. Types of Operations

- 1 **Traversing** – Visiting each element once.
- 2 **Insertion** – Adding a new element.
- 3 **Deletion** – Removing an existing element.
- 4 **Searching** – Finding a specific element.
- 5 **Sorting** – Arranging elements in a particular order.
- 6 **Merging** – Combining two data structures.

3. Importance

- ★ Helps to manipulate and manage data efficiently.
- ★ Used in almost all data structures.
- ★ Essential for algorithm design.
- ★ Improves performance and usability of programs.
- ★ Used in real-world applications like databases, file systems, networks, etc.

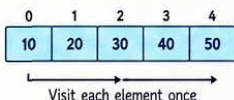
4. Time Complexity (General)

Operation	Typical Time Complexity
Traversing	$O(n)$
Insertion	$O(1)$ or $O(n)$
Deletion	$O(1)$ or $O(n)$
Searching	$O(n)$ or $O(\log n)$
Sorting	$O(n \log n)$ (most common)
Merging	$O(n + m)$

5. Traversing

- Visiting each element of a data structure one by one to process it.
- It allows us to perform some operation (e.g., display, sum) on each element.

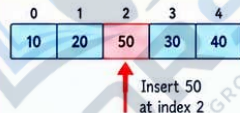
Example: Array Traversal



6. Insertion

- Adding a new element to the data structure at a specified position.
- Different data structures have different ways and time complexities for insertion.

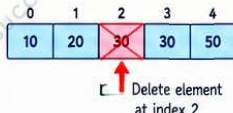
Example: Insert 50 at index 2 in Array



7. Deletion

- Removing an existing element from the data structure.
- Elements after deletion may need to be shifted (in arrays, lists, etc.).

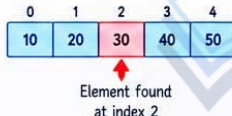
Example: Delete element at index 2



8. Searching

- Finding the position of a specific element in the data structure.
- Can be done using different methods like Linear Search or Binary Search.

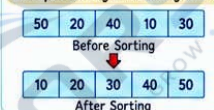
Example: Search 30 in Array



9. Sorting

- Arranging the elements in a specific order (ascending or descending).
- Common sorting techniques:
 - Bubble Sort
 - Selection Sort
 - Insertion Sort (etc.)

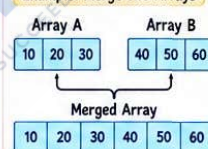
Example: Sorting in Ascending Order



10. Merging

- Combining two or more data structures into one.
- Used in operations like merge sort and in linked lists.

Example: Merge two Arrays



11. Operations on Different Data Structures

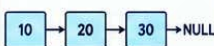
Array

- **Traversing:** Access each element using index.
- **Insertion:** Add at specific position (shift required).
- **Deletion:** Remove element (shift required).
- **Searching:** Linear/Binary.
- **Sorting:** Various sorting algorithms.



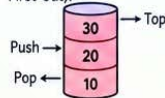
Linked List

- **Traversing:** Move from head to end.
- **Insertion:** At beginning, end or specific node.
- **Deletion:** Remove a node (by adjusting links).
- **Searching:** Linear search.



Stack

- **Traversing:** From top to bottom.
- **Insertion:** Push() – add at top.
- **Deletion:** Pop() – remove from top.
- **Searching:** Linear (if needed).
- **Follows LIFO** (Last In First Out).



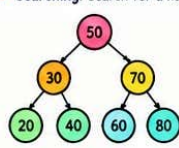
Queue

- **Traversing:** From front to rear.
- **Insertion:** Enqueue() – add at rear.
- **Deletion:** Dequeue() – remove from front.
- **Searching:** Linear (if needed).
- **Follows FIFO** (First In First Out).



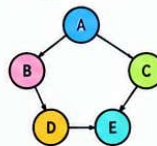
Tree

- **Traversing:** Preorder, Inorder, Postorder.
- **Insertion:** Add a node.
- **Deletion:** Remove a node.
- **Searching:** Search for a key.



Graph

- **Traversing:** BFS, DFS.
- **Insertion:** Add vertex/edge.
- **Deletion:** Remove vertex/edge.
- **Searching:** BFS/DFS.



12. Key Takeaways

- ✓ Data structure operations help in accessing, modifying and managing data.
- ✓ Different data structures support different operations.
- ✓ Traversing, insertion, deletion, searching, sorting and merging are the main operations.
- ✓ These operations are used in various real-world applications.
- ✓ Choosing the right data structure and operation is important for efficient problem solving.



Array is a linear data structure that stores elements of the same type in contiguous memory locations.



Array : Definition and Concepts, Memory Representations

- Key Points:**
- Linear data structure
 - Same data type
 - Contiguous memory
 - Index-based access
 - Fixed size (in C)



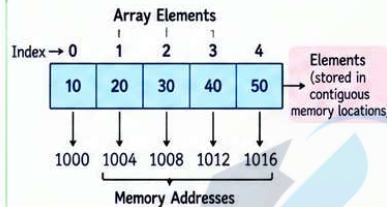
1. What is an Array?

- An array is a linear data structure that stores a collection of elements of the **same data type** in **contiguous (consecutive) memory locations**.
- Each element is accessed using an index (starting from 0).
- Arrays provide a simple and efficient way to store and access multiple values of the same type.

2. Key Concepts of Array

- Homogeneous Elements:** All elements are of the same data type.
- Indexing:** Each element has a unique index (starting from 0).
- Contiguous Memory:** Elements are stored in consecutive memory locations.
- Fixed Size:** In C, the size of an array is fixed and cannot be changed at runtime.
- Direct Access:** Any element can be accessed directly using its index.
- Linear Structure:** Elements are stored in a linear sequence.

3. Representation of Array



4. Terminologies Used

Term	Description
Array	Collection of similar data items.
Element	A single value stored in an array.
Index	Position of an element (starts from 0).
Size	Total number of elements in the array.
Base Address	Address of the first element (index 0).
Contiguous Memory	Elements are stored in consecutive memory locations.
Homogeneous	All elements are of the same data type.

5. Array Declaration (C Language)

```
data_type array_name[size];
```

Example:

```
int a[5]; // An integer array of size 5
float marks[10]; // A float array of size 10
char name[20]; // A character array
```

6. Memory Representation (1D Array)

For an integer array: `int a[5];` (Assume base address = 2000 and each integer occupies 4 bytes)

Index (i)	0	1	2	3	4
Element (a[i])	10	20	30	40	50
Memory Address	2000	2004	2008	2012	2016

Address of $a[i]$ = Base Address + $(i \times \text{size of data type})$

In this example: size of `int` = 4 bytes
So, Address of $a[i]$ = $2000 + (i \times 4)$

7. General Formula

Address of $a[i]$ = $BA + (i \times s)$

Where,

BA = Base Address (address of $a[0]$)
i = Index of the element
s = Size of each element (in bytes)

Example:

If BA = 1000, $i = 3$, $s = 4$ bytes
Then, Address of $a[3]$
= $1000 + (3 \times 4)$
= $1000 + 12$
= 1012



8. Memory Representation (2D Array)

For a 2D array: `int a[2][3];` (Row-major order in C)
Assume base address = 3000 and each integer occupies 4 bytes.

	$a[0][0]$	$a[0][1]$	$a[0][2]$
Row 0	3000	3004	3008
Row 1	3012	3016	3020

Address of $a[i][j]$ = $BA + (i \times \text{number of columns} + j) \times \text{size}$

Here: BA = 3000, number of columns = 3, size = 4 bytes

9. Types of Array (Based on Dimension)

1D Array (One Dimensional)

10	20	30	40	50
----	----	----	----	----

- Single row
- One index (i)
- Example: `int a[5];`

2D Array (Two Dimensional)

11	12	13
21	22	23

- Rows and columns
- Two indices (i, j)
- Example: `int a[2][3];`

3D Array (Three Dimensional)

1	2	3
4	5	6
7	8	9

- Matrix of matrices
- Three indices (i, j, k)
- Example: `int a[2][2][3];`

10. Characteristics of Array

- Stores elements of the same data type.
- Elements are stored in contiguous memory locations.
- Access time is fast ($O(1)$).
- Supports random access using index.
- Fixed size in C (cannot be changed at runtime).
- Memory is allocated at the time of declaration.
- Can be one-dimensional, two-dimensional or multi-dimensional.
- Widely used in real-world applications.

11. Advantages and Disadvantages

Advantages

- Simple and easy to implement.
- Fast access using index ($O(1)$).
- Efficient memory utilization.
- Useful for storing large amounts of similar data.
- Supports multi-dimensional data representation.

Disadvantages

- Fixed size (in C).
- Insertion and deletion can be costly ($O(n)$).
- Stores only similar data types.
- Requires contiguous memory.
- Not suitable for dynamic data where size frequently changes (Use linked list instead).



Array operations help us to process, modify and manage the elements stored in an array.



Array Operations

Traversing

Insertion

Deletion

Operations on Array:
1. Traversing
2. Insertion
3. Deletion



1. What are Array Operations?

- Array operations are basic actions performed on the elements stored in an array.
- The main operations are **Traversing, Insertion and Deletion**.
- These operations help us to **access, modify and manage** the data in an array.
- Arrays have fixed size, so insertion and deletion require shifting of elements.
- These operations are widely used in real-world applications like searching, sorting, and data processing.

2. Traversing an Array

Traversing means visiting each element of an array one by one to process it (e.g., display, sum, etc.).

Steps:

- Start from index 0.
- Access each element using index.
- Perform the required operation on each element.
- Continue until the last element (index $n-1$).

Syntax (C Language)

```
for(int i = 0; i < n; i++) {
    // process a[i]
}
```

Example: Display all elements

Array: `int a[5] = {10, 20, 30, 40, 50};`

Index: 0 1 2 3 4
a[i]: 10 20 30 40 50

C Program:

```
#include <stdio.h>
int main() {
    int a[5] = {10, 20, 30, 40, 50};
    for(int i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

Output:

10 20 30 40 50

Applications of Traversing

- Display all elements.
- Calculate sum/average.
- Find maximum or minimum element.
- Search a specific element.
- Count occurrences of an element.

Time Complexity

Traversing: $O(n)$

Where n is the number of elements in the array.

3. Insertion in an Array

Insertion means adding a new element at a specified position in an array. Since arrays have fixed size, elements after the insertion position are shifted to the right.

Steps:

- Check if the array is full.
- Decide the position to insert.
- Shift all elements to the right from that position.
- Insert the new element at the desired position.

Example: Insert 25 at index 2

Original Array

Index: 0 1 2 3 4
a[i]: 10 20 30 40 50

Step 1: Shift elements to right (from index 2 to 4)

Index: 0 1 2 3 4
a[i]: 10 20 30 40 50

Step 2: Insert 25 at index 2

Index: 0 1 2 3 4
a[i]: 10 20 25 30 40 50

After insertion, array size increases by 1 (if space is available).

C Program: Insert an element

```
#include <stdio.h>
int main() {
    int a[10] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2, item = 25;

    // shift elements to right
    for(int i = n-1; i >= pos; i--) {
        a[i+1] = a[i];
    }

    a[pos] = item;
    n = n + 1;

    printf("Array after insertion: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

Output:

Array after insertion: 10 20 25 30 40 50

Time Complexity of Insertion: $O(n)$

4. Deletion in an Array

Deletion means removing an existing element from a specified position in an array. After deletion, elements after that position are shifted to the left.

Steps:

- Check if the array is empty.
- Decide the position to delete.
- Shift all elements to the left from that position.
- Reduce the size of the array by 1.

Example: Delete element at index 2

Original Array

Index: 0 1 2 3 4
a[i]: 10 20 30 40 50

Step 1: Shift elements to left (from index 3 to 4)

Index: 0 1 2 3 4
a[i]: 10 20 40 50

Step 2: Delete element at index 2

Index: 0 1 2 3
a[i]: 10 20 40 50

After deletion, array size decreases by 1.

C Program: Delete an element

```
#include <stdio.h>
int main() {
    int a[10] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2;

    // shift elements to left
    for(int i = pos; i < n-1; i++) {
        a[i] = a[i+1];
    }

    n = n - 1;

    printf("Array after deletion: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

Output:

Array after deletion: 10 20 40 50

Time Complexity of Deletion: $O(n)$

5. Comparison of Operations

Operation	Purpose	Process	Time Complexity	Effect on Size
Traversing	Visit each element	Access each element once	$O(n)$	No change
Insertion	Add a new element	Shift elements to right	$O(n)$	Increases by 1
Deletion	Remove an element	Shift elements to left	$O(n)$	Decreases by 1

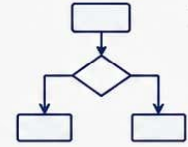
6. Key Takeaways

- Traversing is used to access and process each element.
- Insertion adds a new element and requires right shifting.
- Deletion removes an element and requires left shifting.
- Arrays have fixed size, so insertion and deletion can be costly.
- All these operations have linear time complexity $O(n)$.
- These operations are fundamental for many algorithms.

7. Applications in Real World

- Storing and managing lists of data.
- Used in searching and sorting algorithms.
- Used in image processing (pixel arrays).
- Used in scientific computations (matrix data).
- Used in database records and file handling.

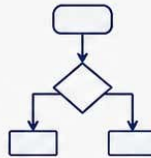
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101





A Stack is a linear data structure that follows the LIFO (Last In First Out) principle.

Stacks : Definition and Concepts, Memory Representations

- Key Points:**
- Linear Data Structure
 - LIFO (Last In First Out)
 - One end access (Top)
 - Push, Pop, Peek
 - Used in many applications



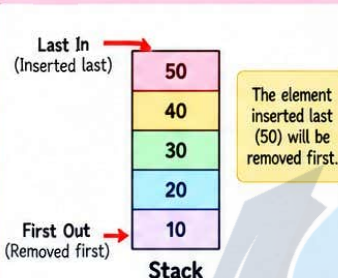
1. What is a Stack?

- A stack is a linear data structure that follows **LIFO (Last In First Out) order**.
- Insertion and deletion are performed only at one end called the **Top**.
- The element inserted last is the first element to be removed.
- Stacks are simple to implement and widely used in real-world applications.
- Examples:** Function calls, Expression evaluation, Undo operations, etc.

2. Key Concepts

- LIFO Principle:** Last element inserted is the first element removed.
- Top:** A pointer that points to the topmost element in the stack.
- Push:** Operation to insert an element at the top.
- Pop:** Operation to remove the topmost element.
- Peek/Top:** Operation to view the topmost element without removing it.
- Stack Overflow:** Occurs when we try to push an element into a full stack.
- Stack Underflow:** Occurs when we try to pop an element from an empty stack.

3. LIFO (Last In First Out)



4. Stack Representation (Conceptual View)

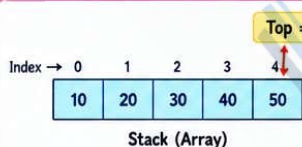


Characteristics:

- Linear data structure
- Follows LIFO order
- Insertion and deletion from top only
- Uses a single pointer (Top)
- Can be implemented using arrays or linked lists
- Fixed size (array implementation) or dynamic size (linked list)

5. Memory Representation using Array

Array Implementation of Stack

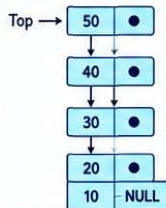


Important Points:

- Stack is implemented using a 1D array.
- Top variable stores the index of the topmost element.
- Initially, Top = -1 (stack is empty).
- When an element is pushed, Top is incremented.
- When an element is popped, Top is decremented.
- Fixed size implementation.

6. Memory Representation using Linked List

Linked List Implementation of Stack



Each node contains:

- Data (value)
- Pointer to next node

Advantages:

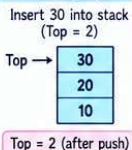
- Dynamic size (no fixed limit)
- Memory used only when required
- Easy insertion and deletion
- at the top

7. Comparison of Representations

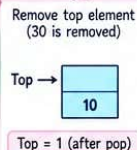
Feature	Array Implementation	Linked List Implementation
Memory allocation	Contiguous memory	Non-contiguous memory
Size	Fixed size	Dynamic size
Implementation	Simple	Slightly complex
Memory utilization	May waste memory	Efficient memory usage
Overflow	Occurs when array is full	Occurs when memory is exhausted
Underflow	Occurs when stack is empty	Occurs when stack is empty

8. Stack Operations (Memory Illustration)

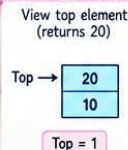
Push(30)



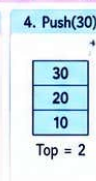
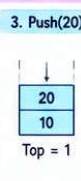
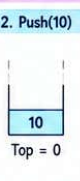
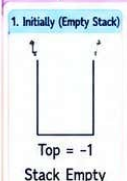
Pop()



Peek()/Top()



9. Operations Illustration (Step-by-Step)



10. Applications of Stack

- Function call management (Recursion)
- Expression evaluation and conversion (Infix, Prefix, Postfix)
- Undo and Redo operations in text editors
- Parenthesis matching
- Backtracking algorithms (e.g., maze, N-Queens)
- Depth-First Search (DFS) in graphs
- Memory management (stack memory)
- Browser history navigation
- Compiler design (syntax checking)

11. Advantages and Disadvantages

Advantages

- Simple and easy to implement
- Efficient for LIFO operations
- Useful in many real-world applications
- Can be implemented using arrays or linked lists

Disadvantages

- Insertion and deletion only at one end
- Fixed size in array implementation
- Stack overflow and underflow issues



Stack operations help us to traverse, insert and delete elements from the stack.

Stack Operations

Traversing

Insertion

Deletion

LIFO Principle:

- Last In First Out
- Insert and delete at Top
- Only the top element can be accessed

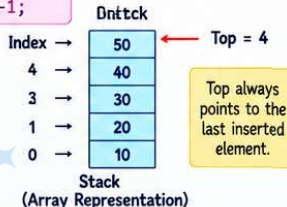


1. What are Stack Operations?

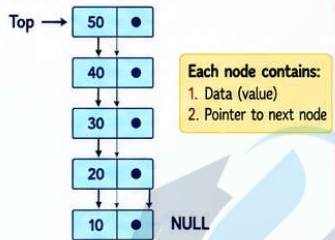
- Stack operations are basic actions performed on the elements of a stack.
- Main operations: **Traversing**, **Insertion (Push)** and **Deletion (Pop)**.
- These operations help us to access, add and remove elements in a stack.
- All operations are performed at the **Top** of the stack.
- Stack follows **LIFO (Last In First Out)** principle.

2. Stack Representation (Array)

```
int stack[MAX];
int top = -1;
```



3. Stack Using Linked List (Concept)



4. Operations at a Glance

Operation	Purpose	Condition	Time Complexity
Traversing	Visit and display all elements	top == -1 (empty)	O(n)
Insertion (Push)	Add a new element at top	top == MAX-1 (full)	O(1)
Deletion (Pop)	Remove the top element	top == -1 (empty)	O(1)

5. Traversing a Stack

Concept

- Traversing means visiting and displaying each element of the stack from top to bottom.
- We start from top and move towards index 0.
- It does not change the stack.
- Time Complexity: O(n)

Algorithm

- Check if stack is empty.
- Start from top and move towards 0.
- Print each element.
- Stop when index becomes -1.

C Program: Traversing a Stack

```
#include <stdio.h>
int main() {
    int stack[5] = {10, 20, 30, 40, 50};
    int top = 4;
    if (top == -1) {
        printf("Stack is empty");
    } else {
        printf("Stack elements (top to bottom):");
        for (int i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        return 0;
    }
}
```

Output:

Stack elements (top to bottom):
50 40 30 20 10

6. Insertion Operation (Push)

Concept

- Insertion in a stack is called Push().
- A new element is always inserted at the top.
- Before insertion, check if the stack is full (overflow).
- After insertion, increment top.

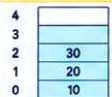
Algorithm (Push)

- Check if stack is full (top == MAX-1).
- If not full, increment top (top++).
- Insert the new element at stack[top].

C Program: Push Operation

```
#include <stdio.h>
#define MAX 5
int main() {
    int stack[MAX], top = -1, item;
    if (top == MAX - 1) {
        printf("Stack Overflow");
    } else {
        item = 25;
        stack[top] = item;
        printf("Inserted %d\n", item);
        return 0;
    }
}
```

Before Push (top = 2)



After Push (top = 3)



7. Deletion Operation (Pop)

Concept

- Deletion in a stack is called Pop().
- It removes the top element.
- Before deletion, check if the stack is empty (underflow).
- After deletion, decrement top.

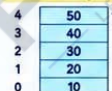
Algorithm (Pop)

- Check if stack is empty (top == -1).
- If not empty, store the top element.
- Decrement top (top--).
- Return/print the deleted element.

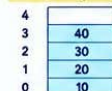
C Program: Pop Operation

```
#include <stdio.h>
int main() {
    int stack[5] = {10, 20, 30, 40, 50};
    int top = 4, item;
    if (top == -1) {
        printf("Stack Underflow");
    } else {
        item = stack[top];
        top--;
        printf("Deleted element: %d\n", item);
        return 0;
    }
}
```

Before Pop (top = 4)



After Pop (top = 3)



8. Menu Driven Program (All Operations)

You can implement push, pop, traverse and peek using a menu:

- Push
- Pop
- Traverse
- Peek
- Exit

9. Time Complexity

Operation	Time Complexity
Traversing	O(n)
Insertion (Push)	O(1)
Deletion (Pop)	O(1)
Peek/Top	O(1)

10. Applications of Stack

- Function call management (recursion).
- Expression evaluation and conversion (infix, prefix, postfix).
- Undo/Redo operations in text editors.
- Backtracking algorithms (maze, N-Queens).
- Browser history navigation.
- Syntax checking in compilers (parenthesis matching).
- Reversing data (strings, arrays).
- Used in many real-world applications.

11. Key Takeaways

- Stack follows LIFO (Last In First Out).
- All operations are performed at the top.
- Traversing visits all elements from top to bottom.
- Insertion (Push) adds a new element at the top.
- Deletion (Pop) removes the top element.
- Use array or linked list for implementation.
- Check for overflow and underflow conditions.
- Push and Pop take O(1) time, Traversing takes O(n).



A Queue is a linear data structure that follows FIFO (First In First Out) principle.



Queue : Definition and Concepts, Memory Representations

- Key Points:**
- Linear data structure
 - Follows FIFO
 - Insertion at Rear
 - Deletion from Front
 - Two pointers: Front and Rear

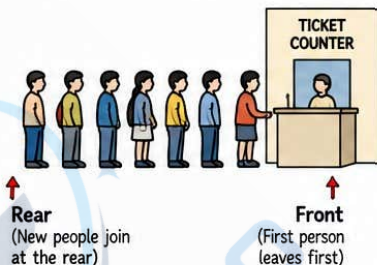


1. What is a Queue?

- A queue is a linear data structure that follows **FIFO (First In First Out)** principle.
- Insertion of elements is done at the rear end (enqueue).
- Deletion of elements is done at the front end (dequeue).
- The element that is inserted first is removed first.
- It works like a real-life queue (e.g., people standing in a line).
- Queues are widely used in operating systems, scheduling, simulations, etc.

2. Real-Life Example

People standing in a queue at a ticket counter.



3. Key Concepts

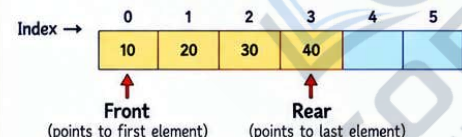
- FIFO (First In First Out):** The first inserted element is the first one to be removed.
- Front:** A pointer that points to the first element of the queue (deletion).
- Rear:** A pointer that points to the last element of the queue (insertion).
- Enqueue:** Operation to add an element at the rear.
- Dequeue:** Operation to remove an element from the front.
- Underflow:** Occurs when the queue is empty and we try to delete an element.
- Overflow:** Occurs when the queue is full (in case of fixed-size array).

4. Basic Operations

Operation	Description	Position	Time Complexity
Enqueue	Insert an element into the queue	At Rear	O(1)
Dequeue	Remove an element from the queue	From Front	O(1)
Front	Get the first element without removing it	At Front	O(1)
Rear	Get the last element without removing it	At Rear	O(1)
IsEmpty	Check if queue is empty	-	O(1)
IsFull	Check if queue is full (in array implementation)	-	O(1)

5. Memory Representation (Array Implementation)

Queue can be implemented using an array of fixed size. We use two indices: front and rear.



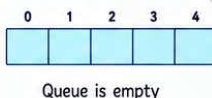
Important Points:

- Elements are stored in contiguous memory locations.
- Front points to the first element. Rear points to the last element.
- Initially, front = -1 and rear = -1 (empty queue).
- Queue is full when rear = size - 1.
- Queue is empty when front = -1 or front > rear.

6. Array Representation with Examples

Initial Queue (Empty)

front = -1 rear = -1



Queue is empty

After Enqueue (10, 20, 30)

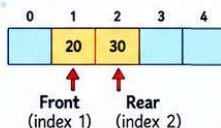
front = 0 rear = 2



Front (index 0) Rear (index 2)

After Dequeue (remove 10)

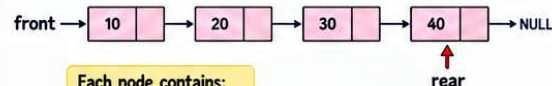
front = 1 rear = 2



Front (index 1) Rear (index 2)

7. Memory Representation (Linked List Implementation)

In linked list implementation, each node contains data and a pointer to the next node. We maintain two pointers: front and rear.



Each node contains:

- Data (value)
- Pointer to next node

8. Node Structure (C Language)

```
struct Node {
    int data;
    struct Node *next;
};
```

Queue Pointers:

```
struct Node *front = NULL;
struct Node *rear = NULL;
```

9. Important Points

- Queue follows FIFO (First In First Out).
- Insertion is done at the rear.
- Deletion is done from the front.
- Two pointers are used: front and rear.
- Can be implemented using array or linked list.
- In array implementation, it has a fixed size.
- In linked list implementation, size is dynamic.
- Used in many real-world applications like scheduling, buffering, etc.

10. Applications of Queue

- CPU scheduling (Process Management).
- Print spooling (Printer queue).
- I/O buffering (Keyboard, Disk, Network).
- Breadth First Search (BFS) in graphs.
- Handling requests in web servers.
- Simulation of real-world scenarios (e.g., ticket counters, call centers).
- Used in operating systems, networks, and asynchronous data transfer.



Traversing a queue means visiting and displaying each element of the queue from the front to the rear without changing its order.



Queue Operations : Traversing

Key Points:

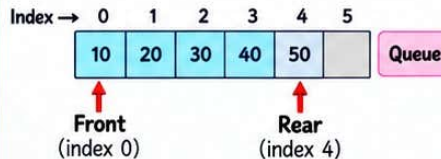
- Follows FIFO (First In First Out)
- Traverse from Front to Rear
- Elements are only accessed, not removed
- Time Complexity: $O(n)$



1. What is Traversing in Queue?

- Traversing means visiting each element of the queue and displaying or processing it one by one.
- It starts from the front element and moves towards the rear element.
- While traversing, elements are **not removed from the queue**.
- The order of elements remains the same (FIFO).
- Traversing is useful to view, print or perform some operation (e.g., sum, search, count) on all elements of the queue.

2. Queue Representation (Array)



Elements in queue (front to rear):
10 → 20 → 30 → 40 → 50

3. Algorithm for Traversing (Array Implementation)

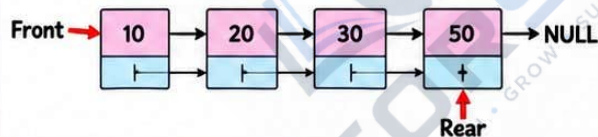
- Check if the queue is empty (front == -1 or front > rear).
- Start from front index.
- Access and display each element.
- Move to the next index (front + 1, front + 2 ...).
- Continue until the rear index is reached.
- Do not modify front or rear during traversal.

Traversing only displays elements, it does not remove them.

Pseudo Code:

```
if queue is empty
    print "Queue is empty"
else
    for i = front to rear
        print queue[i]
```

4. Queue Representation (Linked List)



Each node contains:

- Data (value)
- Pointer to next node

Elements in queue
(front to rear):

10 → 20 → 30 → 40 → 50

5. Algorithm for Traversing (Linked List)

- Check if the queue is empty (front == NULL).
- Start from the front node.
- Access and display the data of current node.
- Move to the next node (temp = temp->next).
- Continue until temp becomes NULL.
- Do not modify the front or rear pointer.

Pseudo Code:

```
if front is NULL
    print "Queue is empty"
else
    temp = front
    while temp != NULL
        print temp->data
        temp = temp->next
```

6. C Program: Traversing a Queue (Array)

```
#include <stdio.h>
#define MAX 10
int queue[MAX], front = -1, rear = -1;

void traverse() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements (front to rear): ");
    for (int i = front; i <= rear; i++) {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main() {
    // Example: queue has 10, 20, 30, 40, 50
    front = 0; rear = 4;
    queue[0] = 10; queue[1] = 20; queue[2] = 30;
    queue[3] = 40; queue[4] = 50;
    traverse();
}
```

Output:

Queue elements
(front to rear):
10 20 30 40 50

7. Applications of Traversing Queue



- ★ Display all elements in a queue.
- ★ Find the sum or average of elements.
- ★ Search for a specific element.
- ★ Count the number of elements.
- ★ Used in scheduling and simulation programs.
- ★ Helpful in debugging and printing queue status.

8. Important Points / Key Takeaways



- ✓ Traversing starts from front and ends at rear.
- ✓ Elements are only accessed, not removed.
- ✓ Works for both array and linked list implementation.
- ✓ Order of elements remains the same (FIFO).
- ✓ Time Complexity: $O(n)$ (where n is the number of elements).
- ✓ Useful for displaying, searching, summing, or processing elements.



Queue operations help us to add (insert) and remove (delete) elements from the queue while maintaining FIFO order.



Queue Operations: Insertion and Deletion

Key Points:

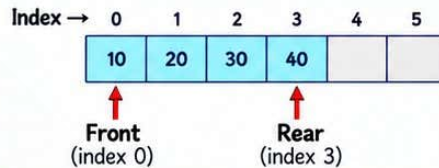
- Follows FIFO (First In First Out)
- Insertion at Rear (Enqueue)
- Deletion from Front (Dequeue)
- Uses two pointers: Front & Rear
- Handles Overflow and Underflow



1. Queue Operations Overview

- A queue supports two main operations:
 - Insertion (Enqueue):** Add a new element at the rear (end) of the queue.
 - Deletion (Dequeue):** Remove an element from the front (start) of the queue.
- Both operations follow **FIFO** (First In First Out).
- Insertion increases the **rear** index.
- Deletion increases the **front** index.
- We need to check for **Overflow** and **Underflow** conditions.

2. Queue Representation (Array)



Elements in queue (front to rear):

10 → 20 → 30 → 40

3. Insertion Operation (Enqueue)

Steps:

- Check if the queue is full (Overflow).
- If not full, increment rear (rear = rear + 1).
- Insert the new element at the rear position.

Overflow Condition (Array):

if rear == MAX - 1
Queue is full (Overflow).

Algorithm (Enqueue)

- if (rear == MAX - 1) → Overflow
- rear = rear + 1
- queue[rear] = item

Example: Insert 50



4. C Program for Insertion (Array Implementation)

```
#include <stdio.h>
#define MAX 5
int queue[MAX], front = -1, rear = -1;

void enqueue(int item) {
    if (rear == MAX - 1) {
        printf("Queue Overflow!\n");
        return;
    }
    if (front == -1)
        front = 0;
    rear = rear + 1;
    queue[rear] = item;
    printf("Inserted %d\n", item);
}
```

Output:

Insert(50)
Inserted 50

5. Deletion Operation (Dequeue)

Steps:

- Check if the queue is empty (Underflow).
- Store the element at front (to return).
- If front == rear (only one element), set front = rear = -1 (queue becomes empty).
- Otherwise, increment front (front = front + 1).
- Return the deleted element.

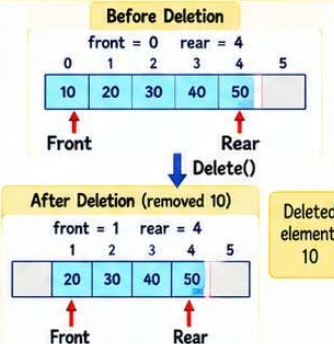
Underflow Condition (Array):

if front == -1
Queue is empty (Underflow).

Algorithm (Dequeue)

- if (front == -1) → Underflow
- item = queue[front]
- if (front == rear)
front = rear = -1
else
front = front + 1
- return item

6. Example: Delete (Dequeue)



Deleted element:
10

7. C Program for Deletion (Array Implementation)

```
int dequeue() {
    if (front == -1) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int item = queue[front];
    if (front == rear) { // only one element
        front = rear = -1;
    } else {
        front = front + 1;
    }
    printf("Deleted %d\n", item);
    return item;
}
```

Output:

Delete()
Deleted 10

8. Applications of Insertion and Deletion in Queue

- Process scheduling (CPU queue).
- Printer queue (print spooling).
- I/O buffering (keyboard, disk, network).
- Handling requests in web servers.
- Simulation of real-world scenarios (call centers, ticket counters).
- Used in operating systems, networks, and asynchronous data transfer.

9. Important Points / Key Takeaways

- Insertion (Enqueue) adds a new element at the rear.
- Deletion (Dequeue) removes an element from the front.
- Follows FIFO (First In First Out) principle.
- Check for Overflow (queue full) and Underflow (queue empty).
- In array implementation, we use two pointers: front and rear.
- Order of elements remains the same.
- Time Complexity: O(1) for both insertion and deletion.



Different types of queues are used to handle elements based on specific requirements, but all follow the basic FIFO (First In First Out) principle.



Types of Queues

Key Points:

- All queues follow FIFO.
- They differ in how insertion and deletion are performed.
- Used for different real-world applications.



1. Linear Queue (Simple Queue)

- The most basic form of queue.
- Insertion is done at Rear and deletion is done at Front.
- Follows FIFO (First In First Out).
- Once the queue is full, no more elements can be inserted even if there is empty space at the front (in array implementation).

Array Representation

0	1	2	3	4
10	20	30	40	

Front
(index 0)

Rear
(index 3)

Operations:

- Enqueue (insert at rear)
- Dequeue (delete at front)

Applications:

- CPU scheduling
- Printer queue
- I/O buffering

2. Circular Queue

- The last position is connected to the first position, forming a circle.
- Solves the problem of unused space in linear queue.
- Insertion is at Rear and deletion is at Front.

Array Representation (Circular)

0	1	2	3	4
50	60	70	80	90

Front
(index 0)

Rear
(index 4)

Operations:

- Enqueue (insert at rear, circularly)
- Dequeue (delete at front, circularly)

Applications:

- Round-robin scheduling
- Buffer management
- Real-time systems

3. Deque (Double Ended Queue)

- Insertion and deletion can be done at both front and rear.
- It can work as both queue and stack.

Array Representation

0	1	2	3	4
10	20	30	40	50

Front
(index 0)

Rear
(index 4)

Operations:

- Insertion at front (enqueueFront)
- Insertion at rear (enqueueRear)
- Deletion from front (dequeueFront)
- Deletion from rear (dequeueRear)

Applications:

- Palindrome checking
- Undo/Redo operations
- Sliding window problems
- Process scheduling

Types of Deque:

- 1 Input Restricted Deque (insertion at only one end)
- 2 Output Restricted Deque (deletion at only one end)

4. Priority Queue

- Each element has a priority value.
- Elements with higher (or lower) priority are removed before others.
- If two elements have the same priority, FIFO is maintained.
- Can be implemented using array, linked list or heap.

Representation (with priority)

1	3	2	5
A	B	C	D

Operations:

- Insert element with priority
- Delete element with highest (or lowest) priority

Types:

- 1 Max Priority Queue (higher priority first)
- 2 Min Priority Queue (lower priority first)

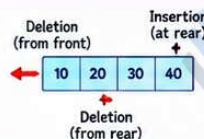
Applications:

- CPU scheduling (priority based)
- Task scheduling
- Network packet handling
- Hospital emergency systems

5. Input and Output Restricted Deque

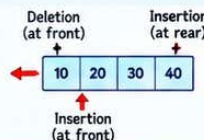
Input Restricted Deque

- Insertion is allowed at only one end (rear).
- Deletion is allowed at both ends (front and rear).



Output Restricted Deque

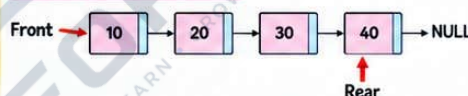
- Deletion is allowed at only one end (front).
- Insertion is allowed at both ends (front and rear).



6. Queue using Linked List

- Each node contains data and a pointer to the next node.
- Insertion is at Rear and deletion is at Front.
- It can grow or shrink dynamically (no fixed size).

Linked List Representation



Operations:

- Enqueue (insert at rear)
- Dequeue (delete at front)

Applications:

- Dynamic memory allocation
- Task scheduling
- Graph algorithms (BFS)

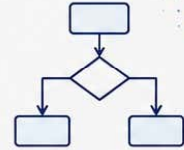
7. Comparison of Queue Types

Queue Type	Insertion	Deletion	Ends Used	Main Feature	Use Cases
Linear Queue	Rear	Front	Two ends (fixed order)	Simple FIFO	CPU, Printer queue
Circular Queue	Rear (circular)	Front (circular)	Two ends (circular)	Reuses empty space	Round-robin, Buffers
Deque	Both ends	Both ends	Both ends	Insertion & deletion at both ends	Undo/Redo, Sliding window
Input Restricted Deque	Rear only	Both ends	Both ends	Insertion at one end	Scheduling, Buffers
Output Restricted Deque	Both ends	Front only	Both ends	Deletion at one end	Specialized scheduling
Priority Queue	According to priority	Highest/lowest priority	Two ends	Elements removed based on priority	CPU scheduling, Networks
Queue (Linked List)	Rear	Front	Two ends (dynamic)	Dynamic size	Dynamic memory, BFS

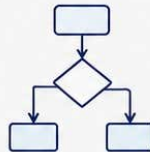
8. Key Takeaways

- ✓ All types of queues follow FIFO (except priority queue which uses priority).
- ✓ Linear queue is simple but may waste space.
- ✓ Circular queue efficiently uses memory.
- ✓ Deque allows insertion and deletion from both ends.
- ✓ Input restricted deque allows insertion at one end only.
- ✓ Output restricted deque allows deletion at one end only.
- ✓ Priority queue removes elements based on priority.
- ✓ Queues can be implemented using arrays or linked lists.
- ✓ Each type of queue is useful in different real-world applications.

1010101010
0101010101
1010101010
0101010101



10110
01001
0101





A linked list is a linear data structure in which elements (nodes) are stored in non-contiguous memory locations and are linked using pointers.



Linked List : Definition and Concepts

Key Points:

- Dynamic data structure
- Non-contiguous memory
- Nodes linked using pointers
- Types: Singly, Doubly, Circular
- Used in many real-world applications

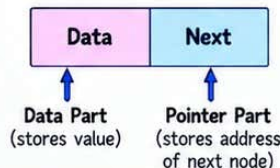


1. What is a Linked List?

- A linked list is a linear data structure in which elements are stored in nodes.
- Each node contains two parts:
 - ✓ Data (value)
 - ✓ Pointer (address of next node)
- Nodes are stored in **non-contiguous** memory locations (unlike arrays).
- Nodes are connected using pointers.
- The first node is called the **head**.
- The last node points to **NULL** (in singly list).

2. Node Structure

Each element in a linked list is called a node.



C Structure of a Node:

```

struct Node {
    int data;
    struct Node *next;
};
  
```

3. Memory Representation

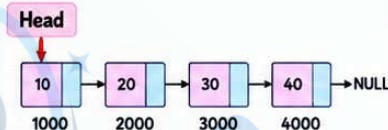
- Nodes are stored in non-contiguous memory locations.
- Each node contains data and a pointer to the next node.

Address	Node	
1000	10	2000
2000	20	3000
3000	30	4000
4000	40	NULL

Memory Locations (Example)

4. How Nodes are Linked?

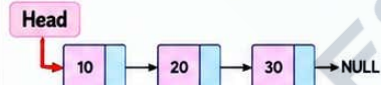
- The pointer in each node stores the address of the next node.
- The last node's next pointer points to NULL (in singly linked list).
- Nodes can be located anywhere in memory.



5. Types of Linked Lists

a) Singly Linked List (SLL)

- Each node contains data and a pointer to the next node.
- Traversal is allowed only in forward direction.
- Last node points to NULL.



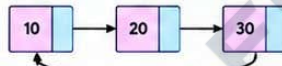
b) Doubly Linked List (DLL)

- Each node contains three parts: Previous pointer | Data | Next pointer.
- Traversal is possible in both forward and backward directions.
- First node's previous points to NULL.
- Last node's next points to NULL.



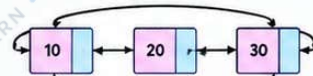
c) Circular Linked List (CLL)

- Last node points back to the first node (instead of NULL).
- Can be singly or doubly circular linked list.
- Useful in applications like round-robin scheduling.



d) Circular Doubly Linked List (CDLL)

- Each node has previous and next pointers.
- Last node's next points to first node and first node's previous points to last node.
- Traversal is possible in both directions in a circular manner.



6. Key Concepts

- Head:** Pointer to the first node of the list.
- Node:** Basic unit containing data and pointer(s).
- Next Pointer:** Points to the next node.
- Previous Pointer (in DLL):** Points to the previous node.
- NULL:** Indicates the end of the list (in SLL and DLL).
- Dynamic Size:** Memory is allocated at runtime as required.

7. Advantages and Disadvantages

Advantages

- ✓ Dynamic size (can grow/shrink).
- ✓ Efficient insertion and deletion (no shifting of elements).
- ✓ Does not require contiguous memory.
- ✓ Useful for implementing stacks, queues, graphs, etc.

Disadvantages

- ✗ Consumes extra memory for pointers.
- ✗ Random access is not possible (must traverse from head).
- ✗ More complex to implement than arrays.
- ✗ Slower access time ($O(n)$ for searching).

8. Applications

- ★ Implementing stacks, queues.
- ★ Dynamic memory allocation.
- ★ Polynomial representation.
- ★ Graph representation (adjacency list).
- ★ Undo/Redo operations (text editors).
- ★ Music playlist (next/previous).
- ★ Browser history (back/forward).
- ★ Used in many real-world systems like operating systems and compilers.



Memory representation shows how the nodes of a linked list are stored in non-contiguous memory locations and linked using pointers.



Linked List : Memory Representations

Key Points:

- Nodes are stored in non-contiguous memory locations.
- Each node contains data and pointer.
- Nodes are linked using addresses.
- Last node points to NULL.



1. Concept of Memory Representation

- In a linked list, each node is stored in a different memory location (non-contiguous).
- Each node contains two parts:
 - Data (value)
 - Pointer (address of next node)
- Nodes are linked using the address of the next node.
- Memory is allocated dynamically at runtime.
- Unlike arrays, linked list nodes do not need to be stored in consecutive memory locations.

2. Structure of Node in Memory



Stores actual value
(e.g. 10, 20, 30)

Stores address of
the next node
(e.g. 2000)

In C, a node is represented as:

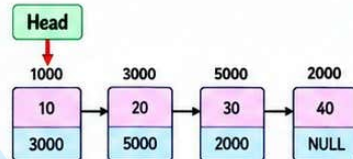
```
struct Node {
    int data;
    struct Node *next;
};
```

3. Memory Representation (Singly Linked List)

Memory Layout

Address	Node Content	
	Data	Next
1000	10	3000
2000	40	NULL
3000	20	5000
4000	(unused)	-
5000	30	2000

Linked List Representation



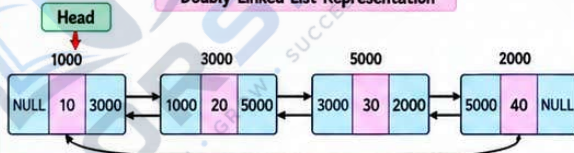
Nodes are stored at random (non-contiguous) memory locations and linked using addresses.

4. Memory Representation (Doubly Linked List)

Memory Layout

Address	Prev	Data	Next
1000	NULL	10	3000
2000	5000	40	NULL
3000	1000	20	5000
5000	3000	30	2000

Doubly Linked List Representation



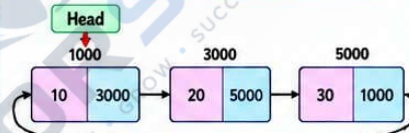
Each node contains a previous pointer and a next pointer. It allows traversal in both forward and backward directions.

5. Memory Representation (Circular Linked List)

Memory Layout

Address	Data	Next
1000	10	3000
3000	20	5000
5000	30	1000

Circular Linked List Representation



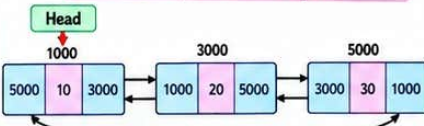
The last node points back to the first node, forming a circular structure.

6. Memory Representation (Circular Doubly Linked List)

Memory Layout

Address	Prev	Data	Next
1000	5000	10	3000
3000	1000	20	5000
5000	3000	30	1000

Circular Doubly Linked List Representation



The last node's next points to the first node and the first node's prev points to the last node.

7. Comparison of Memory Representation

Type	Node Fields	Memory Usage	Connection
Singly Linked List	Data, Next	Less	Next → next node
Doubly Linked List	Prev, Data, Next	More	Prev ↔ next node
Circular Linked List	Data, Next	Less	Last → First
Circular Doubly Linked List	Prev, Data, Next	More	Last → First (Prev & Next)

8. Key Takeaways

- Nodes are stored in non-contiguous memory locations.
- Each node contains data and pointer(s).
- Nodes are linked using addresses.
- Last node points to NULL (in singly and doubly list).
- Circular linked list's last node points to the first node.
- Memory is allocated dynamically.
- Linked list is more flexible than array in memory usage.



Different types of linked lists are used to handle data based on requirements like direction of traversal, number of pointers and structure of connections.



Types of: Linked List

Key Points:

- All are linear data structures.
- Nodes are stored in non-contiguous memory locations.
- Nodes are linked using pointers.
- Each type has different traversal and connection structure.



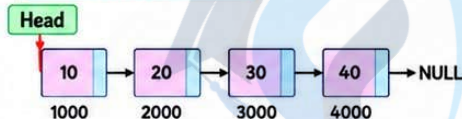
1. Singly Linked List (SLL)

- Each node contains two parts: Data and Next pointer.
- Traversal is possible only in forward direction.
- The last node points to NULL.
- It is the simplest form of linked list.

Node Structure



Memory Representation



Applications:

- Implementation of stacks and queues.
- Dynamic memory allocation.
- Simple data storage and traversal.

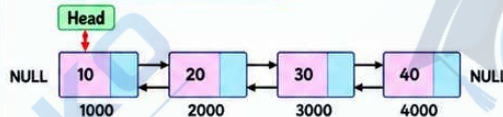
2. Doubly Linked List (DLL)

- Each node contains three parts: Previous pointer, Data and Next pointer.
- Traversal is possible in both forward and backward directions.
- The first node's previous pointer points to NULL.
- The last node's next pointer points to NULL.

Node Structure



Memory Representation



Applications:

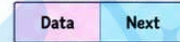
- Undo/Redo operations.
- Browser history (back and forward).
- Music playlist (next and previous).
- Used in advanced data structures.

3. Circular Linked List (CLL)

- The last node points back to the first node (instead of NULL).
- It can be singly circular or doubly circular.
- Traversal can start from any node and continues in a circular manner.
- Useful in round-robin scheduling.

Singly Circular Linked List

Node Structure



Memory Representation

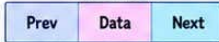


Applications:

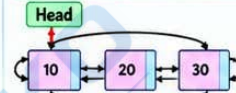
- CPU scheduling (Round Robin).
- Circular buffers.
- Multiplayer games (cyclic iteration).
- Applications where continuous looping is required.

Doubly Circular Linked List

Node Structure



Memory Representation



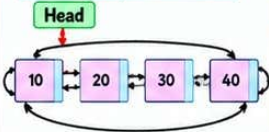
4. Circular Doubly Linked List (CDLL)

- Each node contains three parts: Previous pointer, Data and Next pointer.
- The last node's next pointer points to the first node and the first node's previous pointer points to the last node.
- Traverses in both forward and backward directions in a circular manner.
- It is a combination of doubly and circular linked list.

Node Structure



Memory Representation



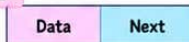
Applications:

- Operating systems (process scheduling).
- Advanced data structures.
- Implementing deques.
- Used in applications requiring bidirectional circular traversal.

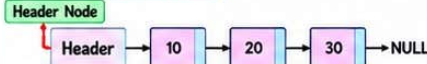
5. Header Linked List

- In this type, an extra node called header node is added at the beginning.
- The header node may or may not contain data.
- It simplifies insertion and deletion operations by providing a fixed starting point.

Node Structure



Memory Representation



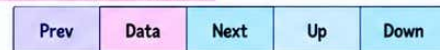
Applications:

- Easier handling of special cases (like empty list).
- Used in file systems and operating systems.
- Common in implementation of other data structures.

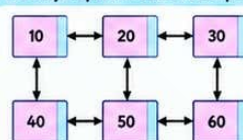
6. Multilinked List (Multiple Linked List)

- Each node can have more than two links (pointers).
- Instead of just next or previous, a node may have multiple pointers (e.g., to next, previous, up, down, etc.).
- It is used to represent complex data structures.

Node Structure (Example)



Memory Representation (Example)



Applications:

- Used in graphs.
- Sparse matrix representation.
- Advanced file systems.
- Multi-level data structures.
- Used where a node needs to be connected in multiple directions.



Traversing a linked list means visiting each node of the list, one by one, to access and process its data.



Linked List Operations: Traversing

Key Points:

- Start from the head node.
- Visit each node and process data.
- Move to the next node using the next pointer.
- Stop when the next pointer becomes NULL.



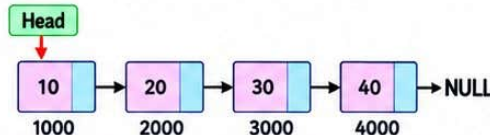
1. Concept of Traversing

- Traversing (or traversal) means visiting each node of the linked list from the first node (head) to the last node.
- During traversal, we access or process the data stored in each node.
- Traversal is used for operations like displaying the elements, searching an element, counting nodes, etc.
- Traversal stops when the next pointer becomes NULL.

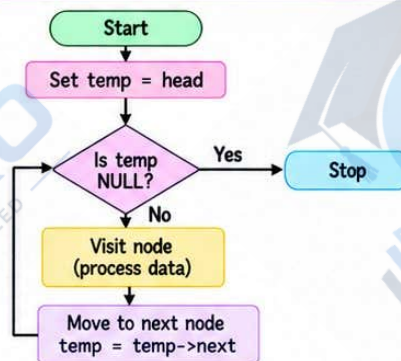
General Algorithm:

- Start from the head node.
- Visit the current node (process data).
- Move to the next node.
- Repeat steps 2 and 3 until the current node becomes NULL.

2. Example Linked List

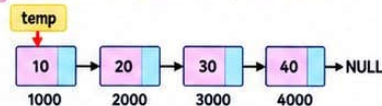


4. Traversing Algorithm (Flowchart)

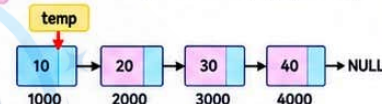


3. Traversal Process (Step by Step)

Step 1: Start with a pointer (temp) at head.



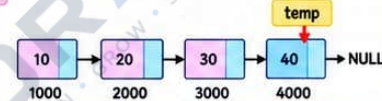
Step 2: Visit the node (process data, e.g. print 10).



Step 3: Move temp to the next node (temp = temp->next).



Step 4: Repeat steps 2 and 3 until temp becomes NULL.



5. C Program to Traverse a Singly Linked List

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};

void traverse(struct Node *head) {
    struct Node *temp = head;
    if (temp == NULL) {
        printf("Linked list is empty!\n");
        return;
    }
    printf("Elements in the linked list: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

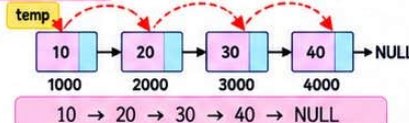
Output:

Elements in the linked list:
10 -> 20 -> 30
-> 40 -> NULL

6. Dry Run / Illustration

Step	temp	Visited Node	Action
1	1000	10	Visit 10, move to 2000
2	2000	20	Visit 20, move to 3000
3	3000	30	Visit 30, move to 4000
4	4000	40	Visit 40, move to NULL
5	N/A	-	Stop traversal

Traversal Order:



7. Applications and Key Takeaways

Applications:

- Display all elements of the list.
- Search for a given element.
- Count the total number of nodes.
- Find the sum or average of elements.
- Used in many algorithms and real-world systems.

Key Takeaways:

- Traversal starts from the head node.
- We use a temporary pointer to move through nodes.
- Each node is visited exactly once.
- Traversal stops when the next pointer is NULL.
- Time complexity: $O(n)$ where n is the number of nodes.
- Traversal is a basic operation used in searching, counting, and displaying.



Insertion means adding a new node to the linked list at a specified position (beginning, end or any given position).



Linked List Operations: Insertion

Key Points:

- A new node is created.
- Memory is allocated dynamically.
- Links (pointers) are adjusted.
- Head may change (if inserted at beginning).
- Time complexity: $O(1)$ at beginning, $O(n)$ at end or at a given position.



1. Concept of Insertion

- Insertion is the operation of adding a new node in a linked list.
- A new node is created with data and next pointer.
- The pointers are adjusted to include the new node in the list.
- Insertion can be done at different positions:
 - At the beginning
 - At the end
 - After a given node
 - At a specific position
- If the list is empty, the new node becomes the head node.

2. Node Creation (Common Step)

- Allocate memory for a new node.
- Assign data to the node.
- Initialize next pointer to NULL.

C Code:

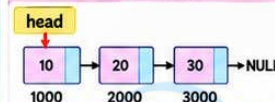
```
struct Node {
    int data;
    struct Node *next;
};

struct Node *newNode;
newNode = (struct Node *)malloc(sizeof(struct Node));
newNode->data = value;
newNode->next = NULL;
```

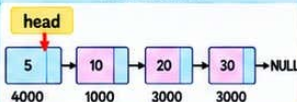
3. Insertion at the Beginning

- A new node is inserted before the current head.
- The new node becomes the head of the list.
- Time complexity: $O(1)$

Before Insertion



After Insertion



Algorithm:

- 1 Create a new node and assign data.
- 2 Set `newNode->next = head`.
- 3 Update `head = newNode`.

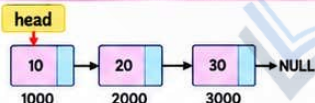
C Code:

```
newNode->next = head;
head = newNode;
```

4. Insertion at the End

- A new node is inserted after the last node.
- Traverse to the last node and link the new node.
- Time complexity: $O(n)$

Before Insertion



After Insertion



Algorithm:

- 1 Create a new node and assign data.
- 2 If list is empty, `head = newNode`.
- 3 Otherwise, traverse to the last node.
- 4 Set `last->next = newNode`.
- 5 `newNode->next = NULL`.

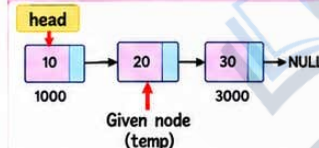
C Code:

```
struct Node *temp = head;
if (head == NULL) {
    head = newNode;
} else {
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}
newNode->next = NULL;
```

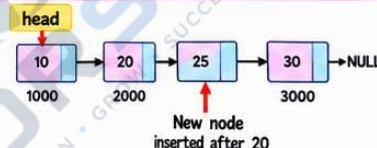
5. Insertion After a Given Node

- A new node is inserted after a specified node (whose address is known).
- Time complexity: $O(1)$ (if the node is given)

Before Insertion



After Insertion



Algorithm:

- 1 Create a new node and assign data.
- 2 Set `newNode->next = temp->next`.
- 3 Set `temp->next = newNode`.

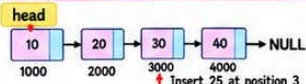
C Code:

```
newNode->next = temp->next;
temp->next = newNode;
```

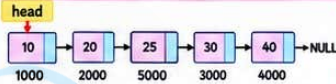
6. Insertion at a Given Position

- A new node is inserted at a specified position (e.g. position 3).
- Traverse to the $(pos - 1)$ th node and insert the new node.
- Time complexity: $O(n)$

Example (Insert at position 3)



After Insertion



Algorithm:

- 1 Create a new node and assign data.
- 2 Traverse to $(pos - 1)$ th node.
- 3 Set `newNode->next = temp->next`.
- 4 Set `temp->next = newNode`.

C Code:

```
struct Node *temp = head;
for (int i = 1; i < pos - 1; i++) {
    temp = temp->next;
}
newNode->next = temp->next;
temp->next = newNode;
```

7. Key Takeaways

- ✓ Insertion adds a new node to the list.
- ✓ Can be done at beginning, end, after a given node, or at a given position.
- ✓ Head changes only when inserting at the beginning (or when list is empty).
- ✓ Uses dynamic memory allocation.
- ✓ Properly adjust the next pointers.
- ✓ Time complexity: $O(1)$ at beginning, $O(n)$ at end or at a given position.
- ✓ Useful in implementing stacks, queues, and dynamic data structures.



Deletion means removing a node from the linked list at a specified position (beginning, end or a given position) and adjusting the links (pointers) to maintain the list.



Linked List Operations : Deletion

Key Points:

- Remove the node and adjust links.
- Memory of deleted node is freed.
- Head may change after deletion.
- Different cases for beginning, end and a given position.
- Time complexity: $O(1)$ or $O(n)$ depending on the position.



1. Concept of Deletion

- Deletion is the operation of removing an existing node from the linked list.
- We first locate the node to be deleted, then adjust the links of adjacent nodes.
- The memory of the deleted node is freed using `free()`.
- Deletion can be done at:
 - At the beginning (first node)
 - At the end (last node)
 - After a given node
 - At a specific position
- If the list is empty, deletion is not possible.

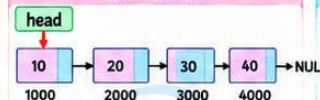
2. General Steps for Deletion

- Check if the list is empty.
- Locate the node to be deleted.
- Adjust the links of the previous node (and/or next node).
- Free the memory of the deleted node.
- Handle special cases (deleting first or last node).

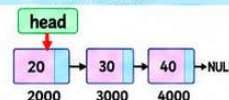
3. Deletion at the Beginning

- Remove the first node (head node).
- Move head to the next node.
- Free the memory of the deleted node.
- Time complexity: $O(1)$

Before Deletion



After Deletion



Algorithm:

- If `head == NULL`, underflow.
- `temp = head;`
- `head = head->next;`
- `free(temp);`

C Code:

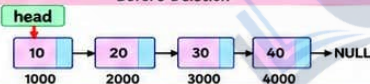
```
struct Node *temp = head;
if (head == NULL) {
    printf("List is empty\n");
} else {
    head = head->next;
    free(temp);
}
```

4. Deletion at the End

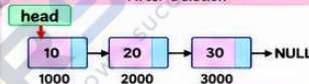
- Traverse the list to find the last node and its previous node.
- Set the next of the previous node to NULL.
- Free the memory of the last node.

Time complexity: $O(n)$

Before Deletion



After Deletion



Algorithm:

- If `head == NULL`, underflow.
- If only one node, set `head = NULL` and free it.
- Otherwise, traverse to the last node keeping track of the previous node.
- Set `prev->next = NULL`.
- Free the last node.

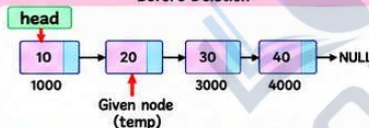
C Code:

```
if (head == NULL) {
    printf("List is empty\n");
} else if (head->next == NULL) {
    free(head);
    head = NULL;
} else {
    struct Node *temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    temp->next = NULL;
    free(temp->next);
}
```

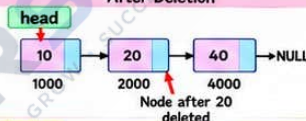
5. Deletion After a Given Node

- Delete the node immediately after a given node (specified node).
- The given node must not be the last node.
- Time complexity: $O(1)$ (if the given node exists)

Before Deletion



After Deletion



Algorithm:

- If `givenNode == NULL` or `givenNode->next == NULL`, deletion is not possible.
- `temp = givenNode->next;`
- `givenNode->next = temp->next;`
- `free(temp);`

C Code:

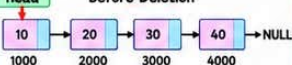
```
if (givenNode == NULL || givenNode->next == NULL) {
    printf("Deletion not possible\n");
} else {
    struct Node *temp = givenNode->next;
    givenNode->next = temp->next;
    free(temp);
}
```

6. Deletion at a Specific Position

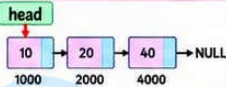
- Delete the node at a given position (e.g. position 3).
- Traverse the list to the $(pos - 1)$ th node and adjust the links.
- Handle special cases like deletion at position 1.
- Time complexity: $O(n)$

Example: Delete node at position 3

Before Deletion



After Deletion



Algorithm:

- If `pos == 1`, delete the first node.
- Otherwise, traverse to $(pos - 1)$ th node.
- If the node to be deleted exists, adjust the links and free it.
- If position is invalid, show an error.

C Code:

```
if (pos == 1) {
    struct Node *temp = head;
    head = head->next;
    free(temp);
} else {
    struct Node *temp = head;
    for (int i = 1; i < pos-1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Invalid position\n");
    } else {
        struct Node *del = temp->next;
        temp->next = del->next;
        free(del);
    }
}
```

7. Key Takeaways

- Deletion removes a node and adjusts the links.
- Must handle special cases (beginning, end, empty list).
- Memory of the deleted node is released using `free()`.
- Time complexity: $O(1)$ at beginning, $O(n)$ at end or a specific position.
- Proper deletion avoids memory leaks and maintains list integrity.



A tree is a hierarchical non-linear data structure consisting of nodes connected by edges. It is used to represent hierarchical relationships.



Tree : Definition and Terminologies

Key Points:

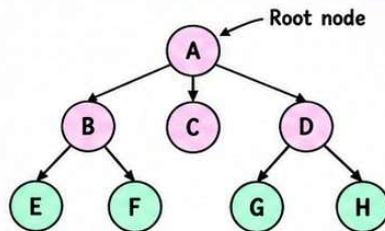
- Non-linear data structure
- Hierarchical representation
- One root node
- Each node has zero or more children
- No cycles
- Used in file systems, databases, compilers, etc.



1. Definition of Tree

- A tree is a finite set of nodes either empty or consists of a root node and a set of disjoint subtrees.
- It is a hierarchical non-linear data structure.
- There is exactly one node called the root node.
- Each node can have zero or more child nodes.
- A tree does not contain cycles.
- It is widely used to represent hierarchical data such as file systems, organizational structures, expression trees, etc.

2. Tree Representation

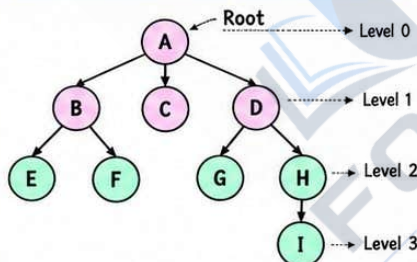


In the above tree, A is the root node, B, C and D are children of A. E, F are children of B and G, H are children of D.

3. Basic Terminologies

1	Node	A node is a basic element of a tree that contains data and links to its children.
2	Root Node	The topmost node of the tree. It has no parent.
3	Parent Node	A node that has one or more child nodes.
4	Child Node	A node that is a descendant of a parent node.
5	Sibling Nodes	Nodes that have the same parent.
6	Leaf Node (External Node)	A node that has no children (degree 0).
7	Internal Node (Non-leaf Node)	A node that has at least one child.
8	Edge	A link or connection between two nodes.
9	Degree of a Node	The number of children a node has.
10	Degree of a Tree	The maximum degree among all nodes in the tree.

4. More Terminologies with Example



Example:

- Root node : A
- Children of A : B, C, D
- Parent of E : B
- Siblings : E and F
- Leaf nodes : E, F, G, I
- Internal nodes : A, B, D, H
- Degree of node B : 2
- Degree of node D : 2
- Degree of node C : 0
- Degree of tree : 2
- Level of node I : 3
- Height of tree : 3

5. Level of a Node

- The level of a node is the length of the path from the root node to that node.
- The root node is at level 0.
- Each child node is at level (parent's level + 1).

Example:

- Level of A = 0
- Level of B, C, D = 1
- Level of E, F, G, H = 2
- Level of I = 3

6. Height of a Node

- The height of a node is the length of the longest path from that node to a leaf node.
- Height of a leaf node is 0.

Example (for above tree):

- Height of I = 0
- Height of E, F, G = 0
- Height of H = 1
- Height of D = 2
- Height of B = 1
- Height of A (root) = 3

7. Depth of a Node

- The depth of a node is the length of the path from the root node to that node.
- Depth of root node is 0.

Example (for above tree):

- Depth of A = 0
- Depth of B, C, D = 1
- Depth of E, F, G, H = 2
- Depth of I = 3

8. Ancestor and Descendant

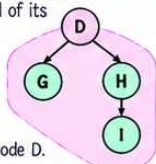
- Ancestor:** A node that lies on the path from the root to the current node.
- Descendant:** A node that lies in the subtree of a given node.

Example:

- Ancestors of I : A, D, H
- Descendants of D : G, H, I

9. Subtree

- A subtree is a tree formed by a node and all of its descendants.



Subtree rooted at node D.

10. Path and Path Length

- Path:** A sequence of nodes connected by edges.
- Path Length:** The number of edges in the path (number of nodes - 1).

Example:

- Path from A to I : A → D → H → I
- Path length = 3 (4 nodes - 1)

11. Summary (Important Points)

- A tree is a hierarchical, non-linear data structure. It has exactly one root node.
- Each node can have zero or more children.
- Nodes are connected by edges.
- There are no cycles in a tree.
- Leaf node has no children.
- Internal node has at least one child.
- Degree of a tree is the maximum degree of any node.
- Level, height and depth help in analyzing tree structure.
- Trees are useful in file systems, databases, compilers, etc.

Memory Representations of Trees

1. What is Memory Representation of a Tree?

- Memory representation refers to how the nodes and their relationships (links) are stored in the computer's memory.
- A tree can be represented in different ways depending on the type of tree and the operations we need to perform.
- The main memory representations are:
 - 1) Linked (Pointer) Representation
 - 2) Sequential (Array) Representation
- Choice of representation affects memory usage, access time and efficiency of operations.

2. Linked (Pointer) Representation

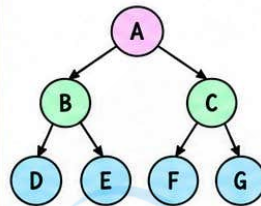
- Each node is stored as a separate record in memory.
- Each node contains data and pointers to its child nodes.
- Widely used for general trees, binary trees, BST, AVL, etc.
- It provides dynamic memory allocation (nodes can be created and deleted at runtime).

Node Structure (Binary Tree)

```
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};
```



3. Example: Linked Representation



Memory Representation

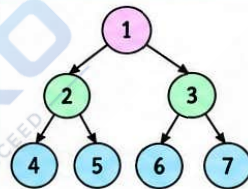
Address	Data	Left	Right
100	A	200	300
200	B	400	500
300	C	600	700
400	D	NULL	NULL
500	E	NULL	NULL
600	F	NULL	NULL
700	G	NULL	NULL

4. Array (Sequential) Representation (For Binary Tree)

- Nodes are stored in a contiguous memory location (array).
- Suitable for complete and almost complete binary trees (e.g. binary heap).
- If a node is stored at index i (starting from 0):

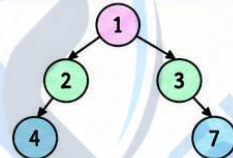
- Left child index = $2i + 1$
- Right child index = $2i + 2$
- Parent index = $(i - 1) / 2$ (for $i > 0$)

5. Example: Array Representation



Index	Value
0	1
1	2
2	3
3	4
4	5
5	6
6	7

6. Array Representation (Non-Complete Tree)



In non-complete trees, some array locations remain empty (waste of space).

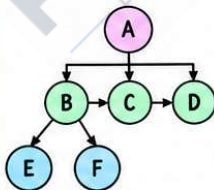
Index	Value
0	1
1	2
2	3
3	4
4	-
5	-
6	7

7. Linked Representation for General Tree

- In a general tree, a node can have any number of children.
- Each node contains data and a list (or pointer) to its children.
- Implemented using a linked list of children.

Node Structure (General Tree)

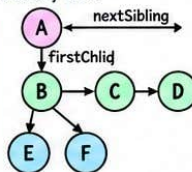
```
struct GNode {
    int data;
    struct GNode *firstChild;
    struct GNode *nextSibling;
};
```



8. Child-Sibling Representation (General Tree)

- Each node has two pointers:
 - firstChild → points to the first child
 - nextSibling → points to the next sibling.
- It converts an n-ary tree into a binary tree.

```
struct Node {
    int data;
    struct Node *firstChild;
    struct Node *nextSibling;
};
```



9. Summary of Memory Representations

Representation	Structure Used	Suitable For	Advantages	Disadvantages
Linked Representation (Binary Tree)	Node with data, left and right pointers	General binary trees, BST, AVL, etc.	Dynamic memory allocation, easy insertion/deletion	Extra memory for pointers, non-contiguous memory
Array Representation	Array	Complete and almost complete binary trees (e.g. heaps)	Simple, fast access, no pointers required	Wastes memory for non-complete trees, fixed size
Linked Representation (General Tree)	Node with list of children (or child-sibling)	General trees (n-ary trees)	Can have any number of children	Complex implementation, extra pointers
Child-Sibling Representation	firstChild and nextSibling pointers	General trees (using binary representation)	Converts n-ary tree to binary tree	More complex, extra pointers

10. Key Takeaways

- Trees can be represented using linked or array representation.
- Linked representation uses pointers and supports dynamic memory allocation.
- Array representation is efficient for complete binary trees.

- For general trees, linked list of children or child-sibling representation is used.
- Choice of representation depends on the type of tree and real-world requirements (efficiency, memory usage, operations).



A tree can be classified into different types based on the number of children, structure and properties of nodes.



Types of Trees

(in Data Structures)

- Key Points:**
- All types of trees are non-linear data structures.
 - Trees are used to represent hierarchical data.
 - Different types of trees are used in different real-world applications.

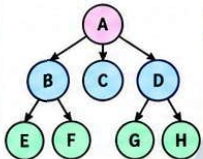


1. General Tree

- A tree in which a node can have any number of children.
- It is the most general form of a tree.

Example / Use:

- File system directory structure.
- Organization hierarchy.

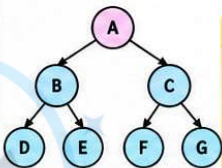


2. Binary Tree

- Each node can have at most two children (left and right).
- Children are ordered (left and right).

Example / Use:

- Expression trees.
- Huffman coding.
- Binary search tree (BST).

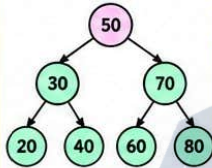


3. Binary Search Tree (BST)

- A binary tree in which for each node:
 - Left subtree contains smaller values.
 - Right subtree contains greater values.

Example / Use:

- Searching, insertion and deletion operations.
- Used in databases, dictionaries, sets.

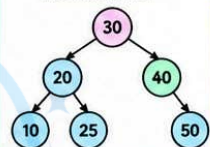


4. AVL Tree (Self-Balancing BST)

- A self-balancing binary search tree.
- The height difference between left and right subtrees is at most 1.
- Maintains balance after insertions and deletions.

Example / Use:

- Used in search operations where balanced height is required.
- Improves efficiency ($O(\log n)$).

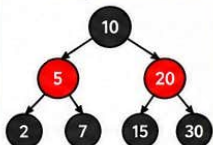


5. Red-Black Tree

- A self-balancing binary search tree with an extra property (node color - red or black).
- Ensures the tree remains balanced during insertions and deletions.

Example / Use:

- Used in STL containers (map, set).
- Widely used in databases and operating systems.

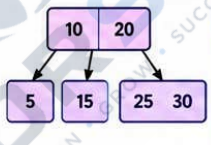


6. B-Tree

- A self-balancing tree in which a node can have more than two children.
- All leaf nodes are at the same level.

Example / Use:

- Used in databases and file systems.
- Efficient for large data stored on disks.

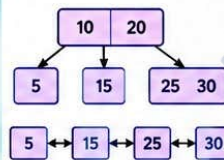


7. B+ Tree

- A variant of B-tree.
- All actual data is stored in leaf nodes.
- Leaf nodes are linked for sequential access.

Example / Use:

- Used in database indexing.
- Faster range queries.
- Widely used in file systems.

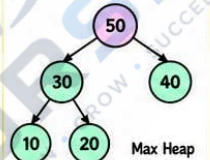


8. Heap (Binary Heap)

- A complete binary tree that satisfies a heap property:
 - Max Heap: parent $\geq$ children.
 - Min Heap: parent $\leq$ children.

Example / Use:

- Used in priority queues.
- Used in heap sort algorithm.
- Efficiently finds minimum or maximum element.

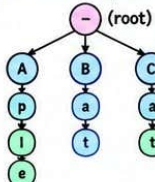


9. Trie (Prefix Tree)

- A tree-like structure used to store strings (characters) rather than values.
- Each node represents a character.

Example / Use:

- Used in auto-completion (like search suggestions).
- Spell checkers, dictionaries.
- IP routing.

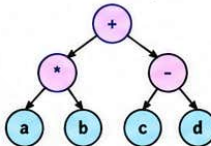


10. Expression Tree

- A binary tree used to represent arithmetic expressions.
- Internal nodes are operators.
- Leaf nodes are operands.

Example / Use:

- Used in compilers to evaluate expressions.
- Converts infix to expression tree.

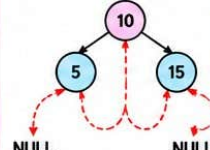


11. Threaded Binary Tree

- A binary tree in which NULL pointers are replaced with special links (threads) to inorder predecessor or successor.

Example / Use:

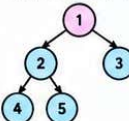
- Used for efficient inorder traversal without stack or recursion.
- Saves time and space.



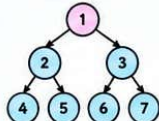
12. Complete and Full Binary Tree

- **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
- **Full Binary Tree:** Every node has either 0 or 2 children.

Complete Binary Tree



Full Binary Tree





A binary tree is a tree data structure in which each node can have at most two children, referred to as the left child and the right child.



Binary Trees

(in Data Structures)

Key Points:

- Each node has at most two children.
- Children are ordered (left and right).
- Can be empty.
- Many important tree types are variants of binary trees (BST, AVL, Heap, etc.).



1. Definition of Binary Tree

- A binary tree is a hierarchical data structure in which each node can have at most two children, called the left child and the right child.
- The children are ordered, i.e., the left child distinct from the right child.
- A binary tree can be empty (no nodes).

Properties:

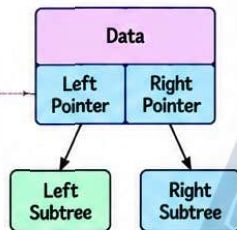
- Maximum 2 children per node.
- Left and right children are different.
- It is a non-linear data structure.
- Can be used to represent hierarchical data.
- Widely used in searching, sorting, expression trees, etc.

2. Structure of a Binary Tree Node

Each node in a binary tree typically contains:

- Data (value)
- Pointer to left child
- Pointer to right child

Node



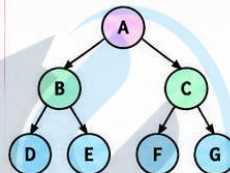
C Structure:

```
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};
```

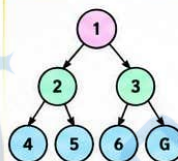
3. Representation of a Binary Tree

- A binary tree is mainly represented using linked (pointer) representation.
- Each node stores data and two pointers (left and right).
- We can also represent a binary tree using an array (mainly for complete binary trees).

Linked Representation



Array Representation (Complete Binary Tree)

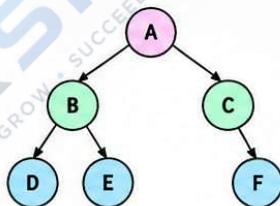


Index	Value
1	1
2	2
3	3
4	4
5	5
6	6
7	7

4. Properties of Binary Trees

- Each node has at most two children.
- Left and right children are ordered.
- The number of nodes at level i can be at most 2^i .
- Maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.
- Minimum number of nodes in a binary tree of height h is $h + 1$.
- A binary tree with n nodes has exactly $n - 1$ edges.
- Empty tree is also a valid binary tree.

5. Example of a Binary Tree



In this tree:

- A is the root.
- B and C are children of A.
- D and E are children of B.
- F is the right child of C.
- Nodes: {A, B, C, D, E, F}
- Root: A
- Leaves: D, E, F
- Internal nodes: A, B, C

6. Types of Binary Trees (Overview)

- Full Binary Tree** – Every node has either 0 or 2 children.
- Complete Binary Tree** – All levels are completely filled except possibly the last level.
- Perfect Binary Tree** – All internal nodes have 2 children and all leaves are at the same level.
- Degenerate (Skewed) Binary Tree** – Each node has only one child (left or right).
- Balanced Binary Tree** – Height is kept limited (e.g., AVL tree).
- Binary Search Tree (BST)** – Left subtree has smaller values, right subtree has greater values.
- Threaded Binary Tree** – NULL pointers are replaced with threads to inorder predecessor/successor.

7. Traversals of a Binary Tree

- Binary tree traversal means visiting all the nodes of the tree in a specific order.
- Main traversal methods:
 - Inorder (Left – Root – Right)
 - Preorder (Root – Left – Right)
 - Postorder (Left – Right – Root)
 - Level Order (Level by level using queue)

8. Applications of Binary Trees

- Expression tree in compilers.
- Representing hierarchical data.
- Searching and sorting (BST).
- Implementing heaps (priority queues).
- Huffman coding (data compression).
- Decision trees in AI and ML.
- File system directories.

9. Advantages and Disadvantages

Advantages

- ✓ Efficient searching (BST).
- ✓ Flexible and dynamic structure.
- ✓ Used in many real-world applications.
- ✓ Can be optimized (balanced trees).

Disadvantages

- ✗ More memory due to pointers.
- ✗ Complex implementation compared to arrays.
- ✗ Not as cache-friendly as arrays.
- ✗ Traversal can be complex for beginners.

10. Key Takeaways

- ✓ A binary tree is a tree where each node has at most two children.
- ✓ Nodes are ordered (left and right).
- ✓ Can be represented using linked list or array.
- ✓ Many important data structures (BST, Heap, AVL) are based on binary trees.
- ✓ Used in searching, sorting, expression evaluation, and real-world systems.



A complete binary tree is a binary tree in which all levels are completely filled except possibly the last level, which is filled from left to right.



Complete Binary Trees

Key Points:

- All levels completely filled except possibly the last level.
- Last level filled from left to right.
- Used in heaps, priority queues.
- Can be represented using array efficiently.



1. Definition

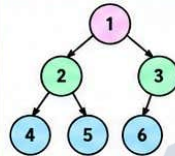
- A complete binary tree is a binary tree in which all levels are completely filled except possibly the last level.
- The nodes in the last level are filled from left to right without any gaps.
- It is a special type of binary tree.
- It may have 0, 1 or 2 children for a node, but the overall structure must satisfy the completeness property.

2. Properties

- 1 All levels except the last level are completely filled.
- 2 The last level is filled from left to right.
- 3 There are no gaps in the last level (i.e., no node to the right of an empty node).
- 4 It contains minimum possible height for a given number of nodes.
- 5 It is always as left as possible.
- 6 A complete binary tree can have between 2^{h-1} and $2^h - 1$ nodes, where h is the height.

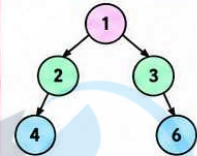
3. Examples

Complete Binary Tree (Valid)



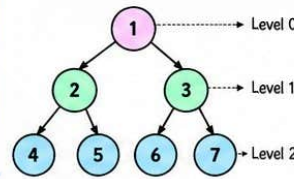
All levels except last are full and last level is filled from left to right.

Not a Complete Binary Tree (Invalid)



Last level is not filled from left to right (gap at node 5).

4. Level wise Representation



In a complete binary tree, nodes are filled level by level from left to right.

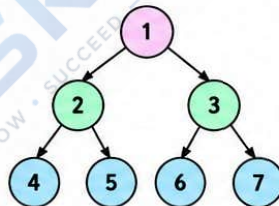
5. Array Representation

- A complete binary tree is efficiently represented using an array.
- If a node is at index i (0-based indexing):

- Left child index = $2i + 1$
- Right child index = $2i + 2$
- Parent index = $(i - 1) / 2$ (for $i > 0$)

- No extra memory is needed for storing links (pointers) because of the complete structure.

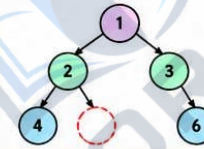
6. Example: Array Representation



Index	Value
0	1
1	2
2	3
3	4
4	5
5	6
6	7

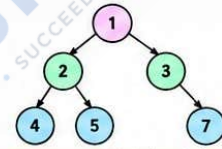
7. Incomplete Cases (Not Complete)

Case 1: Gap in last level (not allowed)



There is a gap at the last level, so it is not a complete binary tree.

Case 2: Last level not filled from left to right



Nodes are not filled from left to right, so it is not complete.

8. Number of Nodes

- For a complete binary tree of height h (0-based level), the number of nodes is:

$$2^h \leq n \leq 2^{h+1} - 1$$

- Minimum number of nodes at height h :

$$n_{\min} = 2^h$$

- Maximum number of nodes at height h :

$$n_{\max} = 2^{h+1} - 1$$

9. Height of a Complete Binary Tree

- The height of a complete binary tree with n nodes is:

$$h = \lfloor \log_2 n \rfloor$$

(0-based indexing)

- Since the tree is complete, its height is minimum compared to other binary trees with the same number of nodes.

10. Applications

- Binary Heap (Min-Heap and Max-Heap).
- Priority Queue.
- Heap Sort algorithm.
- Used in many real-world systems where efficient insertion and deletion are required.
- Memory-efficient representation using arrays.

11. Key Takeaways

- ✓ All levels are completely filled except possibly the last level.
- ✓ Last level is filled from left to right.
- ✓ Efficient array representation.
- ✓ Minimum height for a given number of nodes.
- ✓ Widely used in heaps and priority queues.
- ✓ No gaps allowed in the last level.



A Binary Search Tree (BST) is a binary tree in which for every node, all nodes in the left subtree are smaller and all nodes in the right subtree are greater.



Binary Search Trees (BST)

- Key Points:**
- It is a special type of binary tree.
 - Left subtree < Node < Right subtree
 - Nodes are usually unique.
 - Allows efficient searching, insertion and deletion.
 - Widely used in databases, dictionaries, sets, etc.



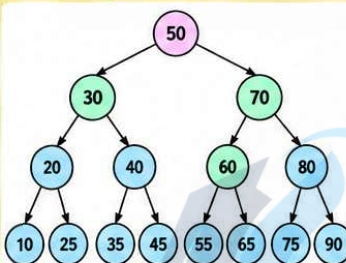
1. Definition

- A Binary Search Tree (BST) is a binary tree in which for every node:
 - All nodes in the left subtree contain values smaller than the node's value.
 - All nodes in the right subtree contain values greater than the node's value.
- The left and right subtrees are also binary search trees.
- Duplicate values are usually not allowed (though some implementations allow them in a specific way).

2. Properties of BST

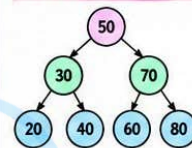
- Left subtree < root < right subtree.
- Both left and right subtrees are also BSTs.
- No duplicate values (in standard BST).
- Inorder traversal gives sorted values (ascending order).
- Searching, insertion and deletion can be done efficiently.
- Average time complexity is $O(\log n)$ for balanced BST and worst case $O(n)$ for skewed BST.

3. Example of a BST



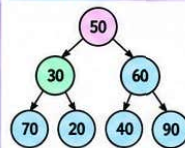
4. BST vs General Binary Tree

Binary Search Tree (BST)



- Left subtree < Node < Right subtree
- Inorder gives sorted order
- Efficient search/insert/delete
- Used in searching and sorting

General Binary Tree



- No ordering property
- Can have any values
- Inorder may not be sorted
- Used for general hierarchical data representation

5. Operations on BST

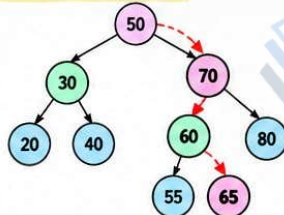
- Searching – Find a value in the tree.
- Insertion – Add a new value while maintaining BST property.
- Deletion – Remove a node while maintaining BST property.
- Traversal – Inorder, Preorder, Postorder.
- Finding minimum/maximum value.
- Finding successor and predecessor.
- Other operations – count nodes, find height, etc.

6. Searching in a BST

Algorithm:

- Start from the root node.
- If the key is equal to the current node → found.
- If the key is smaller → search in left subtree.
- If the key is greater → search in right subtree.
- Repeat until the key is found or reach NULL (not found).

Example: Search 65

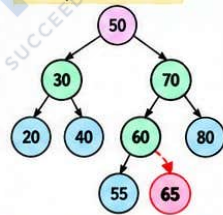


7. Insertion in a BST

Algorithm:

- If the tree is empty, insert the node as root.
- Compare the value with current node.
- If smaller → insert in left subtree.
- If greater → insert in right subtree.
- Repeat until the correct position is found.

Example: Insert 65



8. Deletion in a BST

Cases:

- Node is a leaf (no children) → simply delete it.
- Node has one child → replace the node with its child.
- Node has two children → replace with inorder successor (smallest in right subtree) or inorder predecessor (largest in left subtree).

9. Inorder Traversal of a BST

- Inorder traversal (Left → Root → Right) of a BST always gives nodes in sorted (ascending) order.
- This property is unique to BST.

Example (Inorder of above tree):

10, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 90

10. Applications of BST

- Searching and sorting.
- Implementing dictionaries and sets.
- Symbol table management in compilers.
- Database indexing (e.g., B-Trees are based on the idea of BST).
- Expression evaluation.
- Finding ranges (e.g., all values in a given range).
- Used in file systems, memory management, and many real-world applications.

11. Time Complexity

Operation	Average Case (Balanced BST)	Worst Case (Skewed BST)
Search	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$
Inorder Traversal	$O(n)$	$O(n)$
Find Minimum	$O(\log n)$	$O(n)$
Find Maximum	$O(\log n)$	$O(n)$



Tree Traversal: Preorder



1. What is Tree Traversal?

- Tree traversal means visiting all the nodes of a tree in a systematic order.
- It is used to process, display or perform some operation on each node of the tree.
- There are three main depth-first traversal methods:

1. Preorder (Root - Left - Right)
2. Inorder (Left - Root - Right)
3. Postorder (Left - Right - Root)

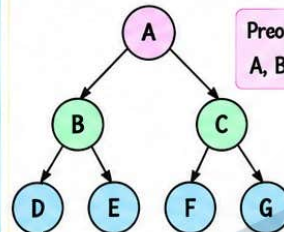
2. What is Preorder Traversal?

- In Preorder traversal, we visit the nodes in the following order:

Root → Left Subtree → Right Subtree

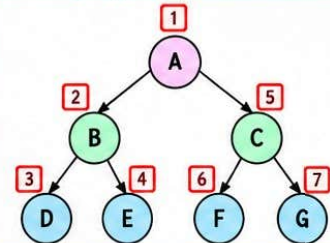
- First the root node is visited.
- Then the left subtree is traversed in preorder.
- Finally the right subtree is traversed in preorder.
- It is also called "NLR" traversal (Node - Left - Right).

3. Example Binary Tree



Preorder Traversal:
A, B, D, E, C, F, G

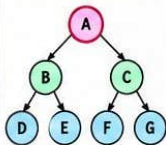
4. Step-by-Step Traversal



Preorder Sequence: A, B, D, E, C, F, G

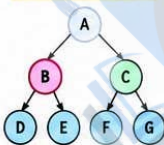
5. Traversal Process

Step 1: Visit A



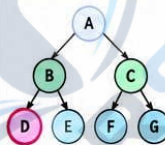
Output: A

Step 2: Traverse left subtree (B)



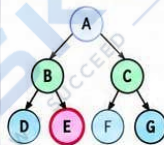
Output: A, B

Step 3: Visit D



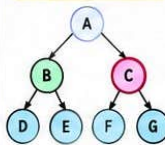
Output: A, B, D

Step 4: Visit E



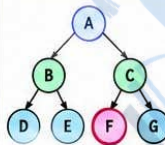
Output: A, B, D, E

Step 5: Traverse right subtree (C)



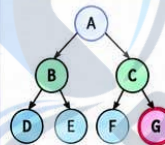
Output: A, B, D, E, C

Step 6: Visit F



Output: A, B, D, E, C, F

Step 7: Visit G



Output: A, B, D, E, C, F, G

6. Algorithm (Recursive)

Preorder(TreeNode root)

- 1 If root is NULL, return.
- 2 Visit the root node (process the data).
- 3 Recursively traverse the left subtree.
- 4 Recursively traverse the right subtree.

7. Pseudocode

Preorder(Node root)

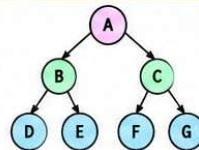
1. if root == NULL
return
2. visit(root)
3. Preorder(root->left)
4. Preorder(root->right)

8. C Program (Recursive)

```

void preorder(struct Node *root) {
    if (root == NULL)
        return;
    printf("%d ", root->data); // Visit root
    preorder(root->left);      // Left subtree
    preorder(root->right);     // Right subtree
}
  
```

9. Output for Example



Preorder Traversal:
A, B, D, E, C, F, G

10. Key Points

- Preorder = Root → Left → Right (NLR).
- Root is visited before its subtrees.
- It is a depth-first traversal method.
- Can be implemented using recursion or stack (iterative).
- Useful in creating a copy of the tree (preorder is used in serialization).
- Time Complexity: $O(n)$
- Space Complexity: $O(h)$ (due to recursion stack)

Tree Traversal: Inorder

1. What is Tree Traversal?

- Tree traversal means visiting all the nodes of a tree in a systematic order.
- It is used to process, display or perform some operation on each node of the tree.
- There are three main depth-first traversal methods:

- Preorder (Root - Left - Right)
- Inorder (Left - Root - Right)
- Postorder (Left - Right - Root)

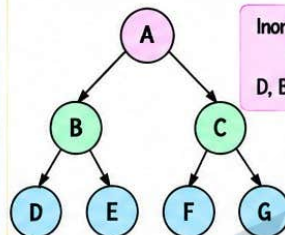
2. What is Inorder Traversal?

- In Inorder traversal, we visit the nodes in the following order:

Left Subtree → Root → Right Subtree

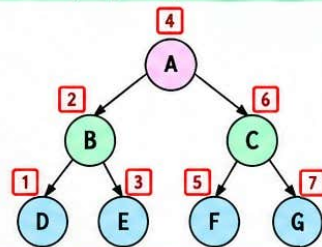
- First the left subtree is traversed in inorder.
- Then the root node is visited.
- Finally the right subtree is traversed in inorder.
- It is also called "**LNR**" traversal (Left - Node - Right).

3. Example Binary Tree



Inorder Traversal
(L, N, R):
D, B, E, A, F, C, G

4. Step-by-Step Traversal

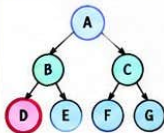


Inorder Sequence: D, B, E, A, F, C, G

5. Traversal Process (Step-by-Step)

Step 1:

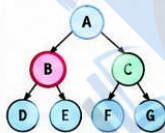
Traverse left subtree of A



Visit D
Output: D

Step 2:

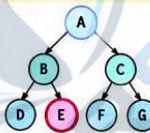
Visit B (after left subtree)



Visit B
Output: D, B

Step 3:

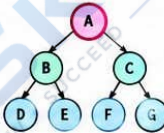
Traverse right subtree of B (Visit E)



Visit E
Output: D, B, E

Step 4:

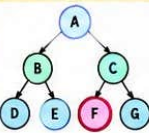
Visit A (after left subtree)



Visit A
Output: D, B, E, A

Step 5:

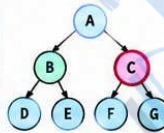
Traverse left subtree of C (Visit F)



Visit F
Output: D, B, E, A, F

Step 6:

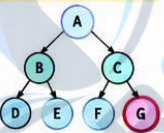
Visit C (after left subtree)



Visit C
Output: D, B, E, A, F, C

Step 7:

Traverse right subtree of C (Visit G)



Visit G
Output: D, B, E, A, F, C, G

6. Algorithm (Recursive)

- If root is NULL, return.
- Recursively traverse the left subtree.
- Visit the root node (process the data).
- Recursively traverse the right subtree.

7. Pseudocode

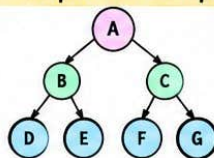
Inorder(root)

- if root == NULL
return
- Inorder(root->left)
- visit(root)
- Inorder(root->right)

8. C Program (Recursive)

```
void inorder(struct Node *root) {  
    if (root == NULL)  
        return;  
    Inorder(root->left); // Left subtree  
    printf("%d ", root->data); // Visit root  
    inorder(root->right); // Right subtree  
}
```

9. Output for Example



Inorder Traversal:
D, B, E, A, F, C, G

10. Key Points

- Inorder = Left → Root → Right (LNR).
- Produces sorted order in a Binary Search Tree (BST).
- Useful for expression trees (used to get infix expression).
- Can be implemented using recursion or stack (iterative).
- Time Complexity: $O(n)$
- Space Complexity: $O(h)$ (due to recursion stack).

Tree Traversal: Postorder

1. What is Tree Traversal?

- Tree traversal means visiting all the nodes of a tree in a systematic order.
- It is used to process, display or perform some operation on each node of the tree.
- There are three main depth-first traversal methods:

- Preorder (Root - Left - Right)
- Inorder (Left - Root - Right)
- Postorder (Left - Right - Root)

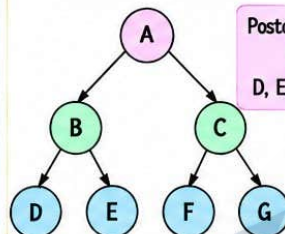
2. What is Postorder Traversal?

- In Postorder traversal, we visit the nodes in the following order:

Left Subtree → Right Subtree → Root

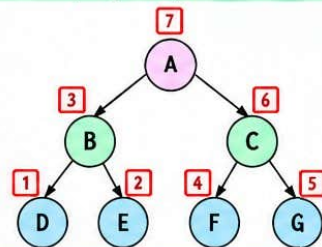
- First the left subtree is traversed in postorder.
- Then the right subtree is traversed in postorder.
- Finally the root node is visited.
- It is also called "**LRN**" traversal (Left - Right - Node).

3. Example Binary Tree



Postorder Traversal
(L, R, N):
D, E, B, F, G, C, A

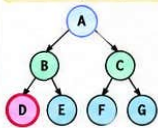
4. Step-by-Step Traversal



Postorder Sequence: D, E, B, F, G, C, A

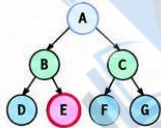
5. Traversal Process (Step-by-Step)

Step 1:
Traverse left subtree of A (Visit D)



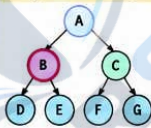
Visit D
Output: D

Step 2:
Visit E (after left subtree)



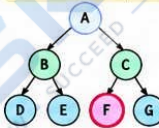
Visit E
Output: D, E

Step 3:
Visit B (after left and right subtree of B)



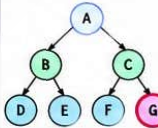
Visit B
Output: D, E, B

Step 4:
Traverse left subtree of C (Visit F)



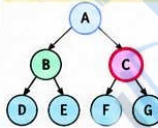
Visit F
Output: D, E, B, F

Step 5:
Visit G (after left subtree)



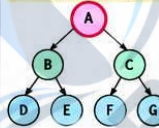
Visit G
Output: D, E, B, F, G

Step 6:
Visit C (after left and right subtree of C)



Visit C
Output: D, E, B, F, G, C

Step 7:
Visit A (after left and right subtree)



Visit A
Output: D, E, B, F, G, C, A

6. Algorithm (Recursive)

- 1 If root is NULL, return.
- 2 Recursively traverse the left subtree.
- 3 Recursively traverse the right subtree.
- 4 Visit the root node (process the data).

7. Pseudocode

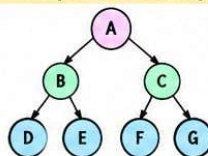
Postorder(root)

1. if root == NULL
return
2. Postorder(root->left)
3. Postorder(root->right)
4. visit(root)

8. C Program (Recursive)

```
void postorder(struct Node *root) {  
    if (root == NULL)  
        return;  
    postorder(root->left); // Left subtree  
    postorder(root->right); // Right subtree  
    printf("%d ", root->data); // Visit root  
}
```

9. Output for Example



Postorder Traversal:
D, E, B, F, G, C, A

10. Key Points

- Postorder = Left → Right → Root (LRN).
- The root is visited after its subtrees.
- It is a depth-first traversal method.
- Useful for deleting a tree (because children are processed before the root).
- Can be implemented using recursion or stack (iterative).
- Used in expression trees (to get postfix expression).
- Time Complexity: $O(n)$
- Space Complexity: $O(h)$ (due to recursion stack).

Insertion in a Binary Search Tree (BST)

1. Definition

- Insertion in a Binary Search Tree means adding a new node with a given value while maintaining the BST property.
- In a BST, for every node:
 - All values in the left subtree are smaller than the node's value.
 - All values in the right subtree are greater than the node's value.
- After insertion, the tree remains a valid BST.

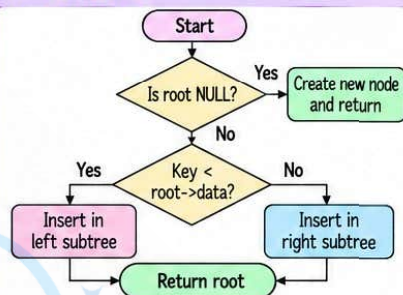
2. Key Points

- Start from the root node.
- Compare the value to be inserted with the current node.
- If the value is smaller, move to the left subtree.
- If the value is greater, move to the right subtree.
- Repeat the process until a NULL position is found.
- Insert the new node at that position.

3. Algorithm (Recursive)

- If the tree is empty ($root == NULL$), create a new node and return.
- If the key is smaller than $root \rightarrow data$, insert in the left subtree.
- If the key is greater than $root \rightarrow data$, insert in the right subtree.
- Return the root node.

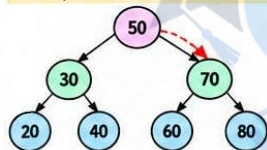
4. Flowchart



5. Example: Inserting a Node (Value = 65)

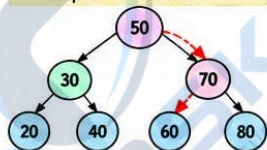
Insert 65 into the following BST:

Step 1: Start at root (50)



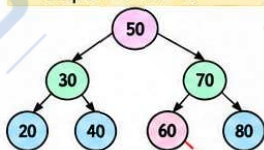
Compare 65 with 50.
Since $65 > 50$, go to right subtree.

Step 2: At node 70



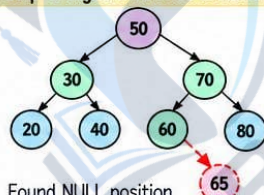
Compare 65 with 70.
Since $65 < 70$, go to left subtree.

Step 3: At node 60



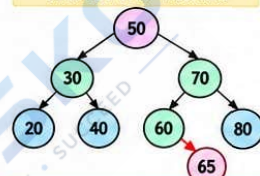
Compare 65 with 60.
Since $65 > 60$, go to right subtree.

Step 4: Right child of 60 is NULL



Found NULL position.
Insert 65 here.

Final BST after Insertion



65 is inserted as the right child of 60.

6. C Program (Recursive Insertion)

```
struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* insert(struct Node* root, int key) {
    if (root == NULL) {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = key;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
    if (key < root->data)
        root->left = insert(root->left, key); // insert in left subtree
    else if (key > root->data)
        root->right = insert(root->right, key); // insert in right subtree
    return root; // return the (unchanged) root node
}
```

7. Iterative Insertion (Idea)

- If tree is empty, create new node.
- Start from root.
- Keep comparing the key with current node.
- Move to left or right subtree accordingly.
- When NULL position is reached, insert the new node there.

8. Time and Space Complexity

Operation	Complexity
Insertion (Average Case)	$O(\log n)$
Insertion (Worst Case)	$O(n)$
Auxiliary Space (Recursive)	$O(h)$

where n is the number of nodes and h is the height of the tree.

9. Key Takeaways

- ✓ Maintain BST property after insertion.
- ✓ Compare the key and move left or right.
- ✓ Insert at the first NULL position.
- ✓ Time complexity is $O(\log n)$ on average.
- ✓ Time complexity becomes $O(n)$ in worst case (skewed tree).
- ✓ Simple and efficient operation.
- ✓ Used in many applications like databases, dictionaries, sets, etc.

Deletion in a Binary Search Tree (BST)

1. Definition

- Deletion in a Binary Search Tree means removing a node with a given value while maintaining the BST property.
- After deletion, the tree must still be a valid BST.
- Three cases are possible depending on the number of children of the node to be deleted:

- Node is a leaf (no children)
- Node has one child
- Node has two children

2. Cases of Deletion

1 Leaf Node (No Children)

- Simply delete the node.
- Set the parent's link to NULL.

2 Node with One Child

- Replace the node with its child.
- Link the parent directly to the child (bypass the node).

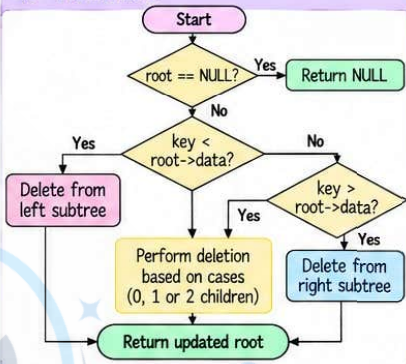
3 Node with Two Children

- Replace the node with its inorder successor (smallest node in right subtree) or inorder predecessor (largest node in left subtree).
- Then delete the inorder successor (predecessor) from the subtree.

3. Algorithm (Recursive)

- If root is NULL, return NULL.
- If key < root->data → go to left subtree.
- If key > root->data → go to right subtree.
- If key == root->data (node found):
 - Case 1: No child → return NULL.
 - Case 2: One child → return the child.
 - Case 3: Two children →
 - Find inorder successor (min in right).
 - Copy its value to current node.
 - Delete the inorder successor from right subtree.
- Return the (possibly updated) root node.

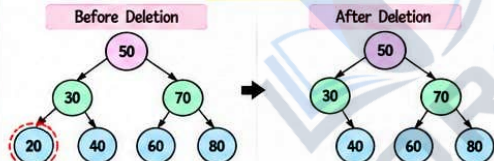
4. Flowchart



5. Examples of Deletion

Case 1: Deletion of a Leaf Node

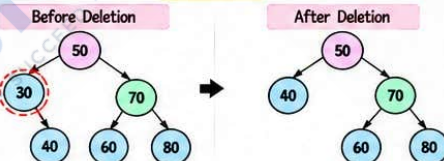
Delete 20



20 is a leaf node, so it is simply removed.

Case 2: Deletion of a Node with One Child

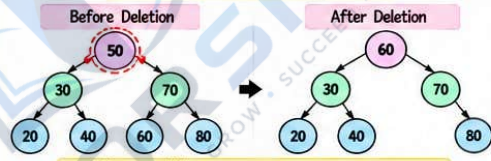
Delete 30



30 has one child (40), so 30 is replaced by its child.

Case 3: Deletion of a Node with Two Children

Delete 50 (using inorder successor)



- 50 has two children.
- Inorder successor = smallest node in right subtree = 60.
- Replace 50 with 60 and delete 60 from right subtree.

6. C Program (Recursive Deletion)

```

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return NULL;
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else { // node found
        // Case 1: No child
        if (root->left == NULL && root->right == NULL) {
            free(root);
            return NULL;
        }
        // Case 2: One child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        // Case 3: Two children
        struct Node* succ = root->right;
        while (succ->left != NULL) // find inorder successor
            succ = succ->left;
        root->data = succ->data; // copy value
        root->right = deleteNode(root->right, succ->data);
    }
    return root;
}
    
```

7. Time Complexity

Operation	Complexity
Deletion (Average Case)	$O(\log n)$
Deletion (Worst Case)	$O(n)$
Auxiliary Space (Recursive)	$O(h)$

where n is the number of nodes and h is the height of the tree.

8. Key Points

- Deletion is done while maintaining BST property.
- Three cases: leaf node, one child, two children.
- For two children, use inorder successor or predecessor.
- Time complexity is $O(\log n)$ on average and $O(n)$ in worst case (skewed tree).
- Can be implemented recursively or iteratively.
- Commonly used in databases, sets, and dynamic data structures.



Searching in a Binary Search Tree (BST)



1. Definition

- Searching in a Binary Search Tree means finding whether a given value (**key**) is present in the tree or not.
- It is the process of comparing the key with the data of nodes starting from the **root** and moving to the left or right subtree based on the BST property.
- If the key is found, we return the node (or success message), otherwise we return **NULL** (or failure message).

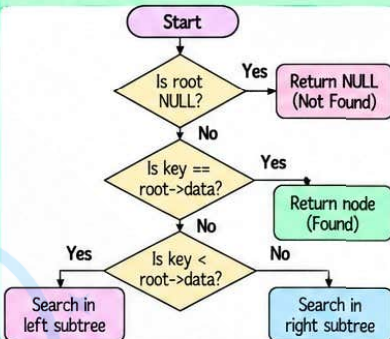
2. Concept / Idea

- A BST stores **smaller** values in the left subtree and **larger** values in the right subtree.
- To search a key, we start from the **root** and compare the **key** with the current node's data.
- If the key is **equal** to the current node's data, the search is successful.
- If the key is **smaller**, we search in the **left subtree**.
- If the key is **larger**, we search in the **right subtree**.
- We repeat this process until we find the key or reach a **NULL** node.

3. Algorithm (Recursive)

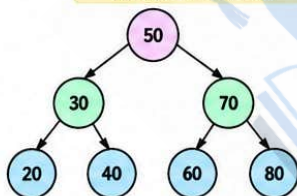
- If root is **NULL**, return **NULL**.
- If $\text{key} == \text{root} \rightarrow \text{data}$, return root (or print found).
- If $\text{key} < \text{root} \rightarrow \text{data}$, search in the **left subtree**.
- If $\text{key} > \text{root} \rightarrow \text{data}$, search in the **right subtree**.

4. Flowchart



5. Step-by-Step Searching Process

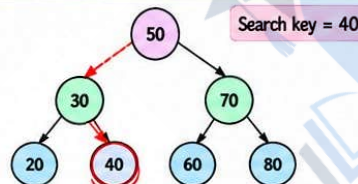
Example: Search key = 40 in the following BST



Steps:

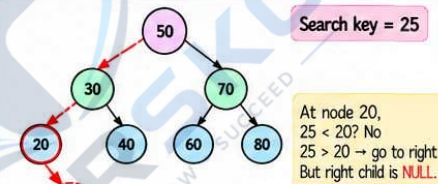
- Start at root (50). Compare 40 with 50.
 $40 < 50 \rightarrow$ go to left subtree.
- At node 30, compare 40 with 30.
 $40 > 30 \rightarrow$ go to right subtree.
- At node 40, compare 40 with 40.
Key found!

6. Example (Key Found)



Output: Key 40 is found.

7. Example (Key Not Found)



Output: Key 25 is not found (NULL).

8. C Program (Recursive Searching)

```

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* search(struct Node *root, int key) {
    if (root == NULL)
        return NULL; // Base case: not found

    if (key == root->data)
        return root; // Found
    else if (key < root->data)
        return search(root->left, key); // Search in left subtree
    else
        return search(root->right, key); // Search in right subtree
}
  
```

9. Time and Space Complexity

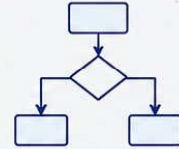
Case	Time Complexity	Explanation
Average Case	$O(\log n)$	Balanced BST (height $\approx \log n$)
Worst Case	$O(n)$	Skewed tree (like linked list)
Space Complexity (Recursive)	$O(h)$	h = height of the tree ($O(\log n)$ average, $O(n)$ worst)

where n is the number of nodes in the tree and h is the height of the tree.

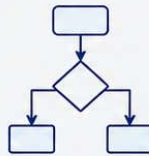
10. Key Points

- ✓ Searching is used to check whether a key is present in the tree.
- ✓ Start from the root node.
- ✓ Compare the key with current node's data.
- ✓ If key is smaller $\rightarrow$ go to left subtree.
- ✓ If key is larger $\rightarrow$ go to right subtree.
- ✓ Repeat until key is found or **NULL** is reached.
- ✓ Time complexity is $O(\log n)$ on average.
- ✓ Time complexity is $O(n)$ in worst case (skewed tree).
- ✓ Space complexity is $O(h)$ due to recursion.

1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - IV

Searching

Definition & Concept



1. Definition of Searching

- Searching is the process of finding whether a given **value (key)** is present in a collection of data (such as an **array**, **list** or **tree**) or not.
- It compares the **key** with the elements of the data structure one by one (or in a specific order) until the element is found or all elements are checked.
- If the element is found, we return its **position (index)** or a success message.
- If the element is **not found**, we return a failure message (such as -1 or **NULL**).

2. Concept of Searching

- Searching is one of the most fundamental operations in computer science.
- It is used to locate a specific element from a collection of data.
- The key idea is to **compare** the **search key** with the elements stored in the data structure.
- The method used for comparison depends on the searching technique (Linear, Binary, etc.).
- The searching process continues until:

- The required element is found, or
- All elements are checked and the element is not found.

3. Search Key (Key Element)

- The value that we want to find in the data structure is called the **search key** (or key element).
- It can be a number, character, string or any other data type.
- Example:** If we want to search for 40 in an array, then **40** is the search key.

Search Key

40

We check if 40 is present in the given data.

4. Examples

Example 1: Searching a number in an array

Array:

0	1	2	3	4	5
10	20	30	40	50	60

Search key = 40

Result: 40 is found at index 3.

Example 2: Searching a string in a list

List:

Apple	Mango	Banana	Orange	Grapes
-------	-------	--------	--------	--------

Search key = Banana

Result: Banana is found at position 2.

5. Types of Searching (Overview)

- There are different searching techniques, depending on **how** the data is arranged.

Searching Techniques

Linear Search
(Sequentially check each element)

Binary Search
(Works on sorted data)

Indexed Sequential Search
(Uses index to speed up searching)

6. Real-Life Analogy



Searching in an unsorted list (Linear Search)

- Suppose you have a pile of books on a table. To find a particular book, you check each book one by one from top to bottom.
- This is similar to **Linear Search**.



Searching in a sorted list (Binary Search)

- Suppose books are arranged in alphabetical order. You directly open the middle shelf and compare the name.
- If it is smaller, you go to the left half, if larger, you go to the right half.
- This is similar to **Binary Search**.

8. Applications of Searching

- Used in databases to quickly find a record.
- Used in dictionaries to find a word.
- Used in search engines to find information.
- Used in file systems to locate a file.
- Used in online shopping to find a product.

9. Advantages and Disadvantages

Advantages

- Helps to quickly find required data.
- Different techniques available for different situations.
- Improves efficiency in data processing.
- Essential in many real-world applications.

Disadvantages

- Some techniques (like binary search) require data to be sorted.
- Linear search is slow for large data ($O(n)$ time).
- Performance depends on the technique used.

10. Summary

Searching is the process of finding a given key element in a collection of data. It works by comparing the key with the elements of the data structure until the element is found or all elements are checked. Different techniques like **Linear Search**, **Binary Search** and **Indexed Sequential Search** are used depending on how the data is arranged.





Searching Techniques



Methods used to find a specific element in a collection of data.



1. Searching Techniques: Overview

- Searching techniques are algorithms used to find whether a **given key** is present in a collection of data or not.
- They compare the **key** with the elements of the data structure one by one or in a specific manner.
- If the element is found, its **position (index)** or a **success message** is returned.
- If the element is **not found**, a failure message (e.g., -1 or **NULL**) is returned.
- Different searching techniques are used depending on **how the data is arranged (sorted or unsorted)** and the size of data.

2. Linear Search

Definition

- Linear Search (Sequential Search) is the simplest searching technique. It checks each element of the list one by one from the beginning until the element is found or all elements are checked.

Working Steps

- Start from the first element.
- Compare the key with the current element.
- If match is found, return the position.
- If no match is found after checking all elements, return failure (-1).

Example



Pseudocode / Algorithm

```

for i = 0 to n-1
    if A[i] == key
        return i // element found
return -1 // element not found

```

Time Complexity

Best Case: $O(1)$
 Average Case: $O(n)$
 Worst Case: $O(n)$

3. Binary Search

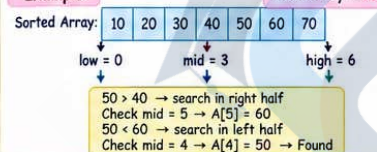
Definition

- Binary Search is an efficient searching technique used **only on sorted data**. It repeatedly divides the search interval into half and compares the key with the middle element.

Working Steps

- Set two pointers: low (start) and high (end).
- Find the middle element $mid = (low + high) / 2$.
- Compare the key with $A[mid]$.
- If $key == A[mid]$, return mid.
- If $key < A[mid]$, search in left half. If $key > A[mid]$, search in right half. Repeat until $low > high$.

Example



Pseudocode / Algorithm

```

low = 0, high = n-1
while (low <= high)
    mid = (low + high) / 2
    if A[mid] == key return mid
    else if key < A[mid] high = mid - 1
    else low = mid + 1
return -1

```

Time Complexity

Best Case: $O(1)$
 Average Case: $O(\log n)$
 Worst Case: $O(\log n)$

4. Indexed Sequential Search

Definition

- Indexed Sequential Search is a searching technique that combines the advantages of **indexing** and **sequential search**. It uses an **index (file)** to quickly locate a block of data, and then performs sequential search within that block.

Working Steps

- Maintain an index file containing key values and pointers to data blocks.
- Search the index file to find the appropriate block.
- Go to the corresponding block in the main file.
- Perform linear (sequential) search within that block.
- If the element is found, return its position.
- If not found, return failure.

Example

Search key = 50

Index file:		Main file (data blocks):		
Key (first element)	Block No.	Block B1:	Block B2:	Block B3:
10	B1	10	20	30
40	B2	40	50	60
70	B3	70	80	90

Pseudocode / Algorithm

- Search the index file for the largest key less than or equal to the search key.
- Go to the corresponding block.
- Perform linear search within that block.
- If found, return the position, else return -1.

Time Complexity

Best Case: $O(1)$
 Average Case: $O(\log n + m)$
 Worst Case: $O(\log n + m)$
 (m = number of records in a block)

5. Comparison of Searching Techniques

Feature	Linear Search	Binary Search	Indexed Sequential Search
Data Requirement	Works on unsorted data	Works only on sorted data	Works on sorted data with index
Method	Sequentially check each element	Divide and search in half	Search in record, then sequential search in block
Time Complexity (Average)	$O(n)$	$O(\log n)$	$O(\log n + m)$
Best Use Case	Small or unsorted data	Large sorted data	Large data stored in files with index
Advantage	Simple, easy to implement	Very fast	More efficient for large file-based data
Disadvantage	Slow for large data	Requires sorted data	More complex, needs extra index structure

6. Real-Life Applications

- Searching in a contact list (Linear Search).
- Searching a word in a dictionary (Linear Search).
- Searching records in a database (Binary Search).
- Searching in a sorted list of roll numbers (Binary Search).
- Searching in large files with index (Indexed Sequential Search), e.g., student records.
- Search engines use indexing techniques (similar to indexed sequential search).
- Finding a product in an online shopping website.

7. Advantages & Disadvantages

Advantages	Disadvantages
- Helps to quickly find a specific element. - Different techniques for different needs. - Binary search is very efficient for large sorted data. - Indexed sequential search is useful for large file systems.	- Linear search is slow for large data. - Binary search requires sorted data. - Indexed sequential search is more complex and needs extra storage for index. - Performance depends on the arrangement and size of data.

8. Key Takeaways

- Searching is used to find a specific element (key) in a collection of data.
- Choose the right technique based on data size, arrangement (sorted/unsorted), and application.
- Linear Search → simple but slow ($O(n)$). Binary Search → fast ($O(\log n)$) but requires sorted data.
- Indexed Sequential Search → combination of indexing and sequential search, useful for large data.



Linear Search



Linear Search is the simplest searching technique that checks each element one by one until the key is found.

1. Definition

- Linear Search (Sequential Search) is a simple searching technique that checks each element of a list/array one by one from the beginning until the required element (key) is found or all elements are checked.
- It compares the search key with each element sequentially.
- If the key is found, it returns the position (index) of the element.
- If the key is not found after checking all elements, it returns a failure message (e.g., -1 or NULL).

2. Working Principle

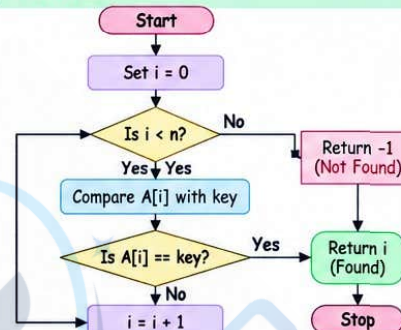
- Start from the first element.
- Compare the key with the current element.
- If the key matches, return the index/position.
- If the key does not match, move to the next element.
- Repeat the process until the key is found or the list ends.
- If the key is not found, return -1 (or a failure message).

3. Algorithm (Step-by-Step)

Input: Array A[0...n-1], key
Output: Position of key or -1

- Step 1: Start
Step 2: For $i = 0$ to $n-1$
Step 3: Compare A[i] with key
Step 4: If A[i] == key, return i
Step 5: If not found, return -1
Step 6: Stop

4. Flowchart



5. Example - Key Found

Search key = 40

	0	1	2	3	4	5
Array:	10	20	30	40	50	60

- Compare 10 with 40 → not equal.
- Compare 20 with 40 → not equal.
- Compare 30 with 40 → not equal.
- Compare 40 with 40 → equal.

Result: Key 40 is found at index 3.

6. Example - Key Not Found

Search key = 25

	0	1	2	3	4	5
Array:	10	20	30	40	50	60

- Compare 10 with 25 → not equal.
- Compare 20 with 25 → not equal.
- Compare 30 with 25 → not equal.
- Compare 40 with 25 → not equal.
- Compare 50 with 25 → not equal.
- Compare 60 with 25 → not equal.

Result: Key 25 is not found (return -1).

7. Pseudocode and C Program

Pseudocode

```

procedure linearSearch(A, n, key)
  for i = 0 to n - 1
    if A[i] == key then
      return i // element found
    end if
  end for
  return -1 // element not found
end procedure
  
```

C Program

```

#include <stdio.h>
int linearSearch(int arr[], int n, int key) {
  for(int i = 0; i < n; i++) {
    if(arr[i] == key)
      return i; // element found
  }
  return -1; // element not found
}

int main() {
  int arr[] = {10, 20, 30, 40, 50, 60};
  int n = sizeof(arr)/sizeof(arr[0]);
  int key = 40;
  int pos = linearSearch(arr, n, key);
  if(pos != -1)
    printf("Key %d found at index %d\n", key, pos);
  else
    printf("Key %d not found\n", key);
  return 0;
}
  
```

8. Time and Space Complexity

Case	Time Complexity	Explanation
Best Case	$O(1)$	Key is found at the first element.
Average Case	$O(n)$	Key may be at any position.
Worst Case	$O(n)$	Key is at the last element or not present.

Space Complexity: $O(1)$ (uses only a few variables)

Where n is the number of elements in the array.



9. Key Points, Advantages and Disadvantages

Key Points

- Linear Search checks each element one by one.
- It works on both sorted and unsorted data.
- Simple to understand and implement.
- Returns the index of the element if found, otherwise -1.
- Time complexity is $O(n)$.

Advantages

- Very simple algorithm.
- Easy to implement.
- Works on both sorted and unsorted data.
- Suitable for small datasets.
- Does not require any special data structure.

Disadvantages

- Time complexity is $O(n)$, which is slow for large data.
- Not suitable for large datasets.
- It compares each element sequentially, so it takes more time.
- Less efficient than Binary Search (for sorted data).

1. Definition

- Binary Search is an **efficient** searching technique used to find a given key in a **sorted list** (array).
- It repeatedly **divides** the search interval into **half** and compares the key with the **middle element**.
- If the key matches the middle element, the search is successful.
- If the key is **smaller**, we search in the **left half**.
- If the key is **larger**, we search in the **right half**.
- If the key is **not found**, it returns a **failure message** (e.g., -1 or NULL).

2. Working Principle

- The array must be sorted.
- Find the middle element of the current search range.
- Compare the key with the middle element.
- If key == middle element, return the position.
- If key < middle element, search in the left half.
- If key > middle element, search in the right half.

Binary Search

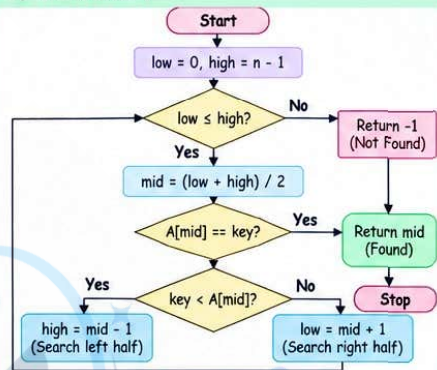
3. Algorithm (Step-by-Step)

Input: Sorted array A[0...n-1], key

Output: Position of key or -1

- Set low = 0 and high = n - 1.
- Calculate mid = (low + high) / 2.
- If A[mid] == key, return mid.
- If key < A[mid], set high = mid - 1 (search in left half).
- If key > A[mid], set low = mid + 1 (search in right half).
- Repeat steps 2 to 5 until low > high.
- If key is not found, return -1.

4. Flowchart



5. Example - Key Found

Step 1: low = 0, high = 6
mid = (0 + 6) / 2 = 3

0	1	2	3	4	5	6
10	20	30	40	50	60	70

mid

A[mid] = 40
which is equal to key.
Key found at index 3.

Step 2:
(Not needed as key found in step 1)

Result: Key 40 is found
at index 3.

6. Example - Key Not Found

Step 1: low = 0, high = 6
mid = 3

0	1	2	3	4	5	6
10	20	30	40	50	60	70

mid

25 < 40
Search in left half.
Set high = 2.

Step 2: low = 0, high = 2
mid = 1

0	1	2	3	4	5	6
10	20	30	40	50	60	70

mid

25 > 20
Search in right half.
Set low = 2.

Step 3: low = 2, high = 2
mid = 2

0	1	2	3	4	5	6
10	20	30	40	50	60	70

mid

25 < 30
Search in left half.
Set high = 1.

Step 4:
Now low = 2, high = 1
(low > high)

Result:
Key 25 is **not** found
(return -1).

7. Pseudocode

```

function binarySearch(A, n, key)
    low = 0
    high = n - 1
    while low <= high
        mid = (low + high) / 2
        if A[mid] == key
            return mid
        else if key < A[mid]
            high = mid - 1
        else
            low = mid + 1
    return -1
  
```

8. C Program

```

#include <stdio.h>
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key) // key found
            return mid;
        else if (key < arr[mid]) // left half
            high = mid - 1;
        else // right half
            low = mid + 1;
    }
    return -1; // key not found.
}

int main() {
    int arr[] = {10, 20, 30, 40, 50, 60, 70};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 40;
    int pos = binarySearch(arr, n, key);
    if (pos == -1)
        printf("Key %d not found\n", key);
    else
        printf("Key %d found at index %d\n", key, pos);
    return 0;
}
  
```

9. Time and Space Complexity

Case	Time Complexity	Explanation
Best Case	O(1)	Key is found at the middle element in first comparison.
Average Case	O(log n)	Search space is reduced by half in each step.
Worst Case	O(log n)	Key is at the last position or not present.

Space Complexity: O(1)

(No extra space is used, only a few variables).

10. Advantages and Disadvantages

Advantages	Disadvantages
✓ Very fast compared to Linear Search.	✗ Works only on sorted data.
✓ Time complexity is O(log n).	✗ Not suitable for unsorted datasets.
✓ Requires fewer comparisons.	✗ More complex than Linear Search.
✓ Efficient for large datasets.	✗ Not efficient for small datasets (Linear Search may be better).
✓ Widely used in real-world applications.	

11. Key Points

- Binary Search works only on sorted arrays.
- It divides the search space into half in each step.
- It compares the key with the middle element.
- If key is smaller → search left half, if larger → search right half.
- Time complexity is O(log n) and space complexity is O(1).



Indexed Sequential Search



1. Definition

- Indexed Sequential Search is a searching technique that combines the advantages of **indexing** and **sequential search**.
- An **index file** is created, which stores **key values** and **pointers** to the main data blocks (file).
- First, the **index is searched** (using binary or sequential search) to locate the **appropriate block**.
- Then, a **sequential search** is performed within that block to find the exact record.
- It is useful for **large files** that are stored in **sorted order**.

2. Concept

- The main file (data file) is divided into **blocks** (also called **segments**).
- An **index file** is created that contains the **first key** (or some representative key) of each block and a **pointer** to that block.
- To search a key, first search the index to find the correct block.
- Then perform a sequential search within that block to find the key.
- Thus, it reduces the number of comparisons compared to a simple sequential search.

3. Structure of Indexed Sequential Search

Index File		Main File (Data File)			
Index Key	Pointer (Block No.)	Block B1	10	20	30
10	B1	Block B2	40	50	60
40	B2	Block B3	70	80	90
70	B3	Block B4	100	110	120
100	B4				

Index file stores the first key of each block and a **pointer** to that block in the main file.

4. Working Steps

- Search the index file to find the largest index key less than or equal to the search key.
- Follow the pointer to the corresponding block in the main file.
- Perform a sequential search within that block to find the key.
- If the key is found, return its position (or record).
- If the key is not found in that block, return a failure message (e.g., -1 or NULL).

5. Example

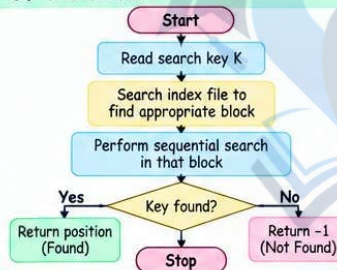
Index File		Main File (Data File)			
Index Key	Pointer	Block B1	10	20	30
10	B1	Block B2	40	50	60
40	B2	Block B3	70	80	90
70	B3	Block B4	100	110	120
100	B4				

Search key = 70

- Search index: Largest index key ≤ 70 is 70 $\rightarrow$ Block B3.
- Go to Block B3 (70 80 90).
- Perform sequential search in B3: compare 70 $\rightarrow$ found.

Result: Key 70 is found in Block B3 (position 1 in block).

6. Flowchart



7. Pseudocode

- Read search key K
- Search index file to find the largest index key $\leq K$
- Get the pointer to the corresponding block
- Perform sequential search within that block
- If key is found return position
- Else return -1
- Stop

8. C Program (Ilimplified Illustration)

```

#include <stdio.h>
int main() {
    int indexKey[] = {10, 40, 70, 100};
    int blockData[][3] = {{10, 20, 30}, {40, 50, 60}, {70, 80, 90}, {100, 110, 120}};
    int nindex = 4, blockSize = 3, key, i, j, block = -1;
    printf("Enter search key: ");
    scanf("%d", &key);
    // Search in index file
    for(i = 0; i < nindex; i++) {
        if(indexKey[i] <= key) block = i;
        else break;
    }
    if(block == -1) { printf("Key not found\n"); return 0; }
    // Sequential search in block
    for(j = 0; j < blockSize; j++) {
        if(blockData[block][j] == key) {
            printf("Key %d found in Block %d at position %d\n", key, block+1, j+1);
            return 0;
        }
    }
    printf("Key not found\n");
    return 0;
}
  
```

9. Advantages and Disadvantages

Advantages

- More efficient than simple sequential search.
- Reduces the number of comparisons by using an index.
- Suitable for large files.
- Access time is less, as only one block needs to be searched.
- Works well when data is stored in sorted order.
- Widely used in database and file systems.

Disadvantages

- Requires extra storage space for the index file.
- Index file must be maintained whenever data is updated.
- More complex than linear or binary search.
- If the index file is not organized properly, performance may degrade.
- Not as fast as Binary Search for in-memory data.
- Works best only on sorted files.

10. Key Points and Applications

Key Points

- It is a combination of indexing and sequential search.
- The index contains key values and pointers to blocks.
- First search the index, then perform sequential search in the block.
- It reduces search time for large datasets.
- Data must be stored in sorted order.
- If the key is not found, it returns -1 or NULL.

Applications

- Used in file systems for efficient record retrieval.
- Used in database management systems.
- Used in libraries and large record management.
- Suitable for large sorted data files.



Sorting

Definition and Concept



Sorting is the process of arranging data in a particular order (**ascending** or **descending**) to make it easier to search, access and manage.

1. Definition of Sorting

- Sorting is the process of arranging the elements of a collection (such as an array or list) in a specific order, either **ascending** (smallest to largest) or **descending** (largest to smallest).
- It reorders the data based on a given key (value) using a **comparison** between elements.
- The main aim of sorting is to make data **organized**, which helps in **faster searching**, **easier access** and **better data management**.

2. Concept of Sorting

- Sorting works by **comparing** two elements and arranging them in the correct order.
- It repeatedly compares and **swaps** (if necessary) the elements until the entire collection is in order.
- Different sorting algorithms use **different techniques** to compare elements and arrange them.
- Sorting can be done in two ways: **Ascending Order** and **Descending Order**.

3. Example of Sorting

Ascending Order (Smallest to Largest)

Unsorted Data:

40 10 50 20 30

Sorted Data (Ascending):

10 20 30 40 50

Descending Order (Largest to Smallest)

Unsorted Data:

40 10 50 20 30

Sorted Data (Descending):

50 40 30 20 10

4. Why is Sorting Needed?

- 1 Makes data organized and structured.
- 2 Helps in **faster searching** (e.g., Binary Search).
- 3 Makes data analysis easier.
- 4 Useful in database systems for record management.
- 5 Helps in removing duplicates.
- 6 Used in real-world applications such as student records, product lists, etc.



5. Types of Sorting Order

Ascending Order

- Elements are arranged from smallest to largest.
- Example: 10, 20, 30, 40, 50
- Also called increasing order.



Descending Order

- Elements are arranged from largest to smallest.
- Example: 50, 40, 30, 20, 10
- Also called decreasing order.



6. Basic Operations in Sorting

- 1 **Comparison** - Compare two elements to decide their correct order.
- 2 **Swap** - If elements are in wrong order, we swap them.
- 3 **Repeat** - Continue the comparison and swapping until the whole list is sorted.
- 4 **Termination** - Stop when no more swaps are needed (array is in order).

7. Real-Life Analogy

- Sorting is similar to arranging books on a shelf.
- If books are arranged in alphabetical order (A, B, C, ...), it becomes easier to find a particular book.
- Similarly, if numbers or names are sorted, we can quickly locate a specific item.



8. Sorting Techniques (Overview)

- There are different sorting algorithms, each using a different method to arrange the data.

Sorting Techniques

Bubble Sort

Repeatedly compares adjacent elements.

Selection Sort

Finds the smallest element and places it in the correct position.

Insertion Sort

Builds the sorted list by inserting elements at the correct position.

9. Advantages and Disadvantages of Sorting

Advantages

- ✓ Helps in faster searching.
- ✓ Organizes data in a meaningful way.
- ✓ Useful in many real-world applications (databases, records, etc.).
- ✓ Makes duplicate removal easier.
- ✓ Improves efficiency in data processing.
- ✓ Helps in using other algorithms like Binary Search, Merge, etc.

Disadvantages

- ✗ Some sorting algorithms are slow for large data (e.g., Bubble Sort).
- ✗ Requires extra time and computation.
- ✗ Some algorithms need extra memory space.
- ✗ Performance depends on the type of algorithm and input data.
- ✗ Not always suitable for very large or real-time data without optimization.

10. Key Takeaways

- ★ Sorting arranges data in a specific order (ascending or descending).
- ★ It works by comparing and swapping elements.
- ★ Major sorting techniques: Bubble Sort, Selection Sort, Insertion Sort.
- ★ Sorting makes data organized, easy to access and efficient to manage.
- ★ It is widely used in real-world applications.



Searching Techniques



Methods used to find a specific element in a collection of data.

1. What are Searching Techniques?

- Searching techniques are algorithms used to find whether a **given element (key)** is present in a collection of data or not.
- They compare the search key with the elements of the data structure and return the **position (index)** if found.
- If the element is **not found**, a failure message (e.g., **-1** or **NULL**) is returned.
- Different searching techniques are used depending on **how the data is arranged (sorted or unsorted)** and the size of the data.

2. Types of Searching Techniques

Searching Techniques

1. Linear Search (Sequential Search)

Works on unsorted data
Checks each element one by one.

2. Binary Search

Works on sorted data
Divides the search space into half.

3. Indexed Sequential Search

Works on sorted data with index
Combines advantages of indexing and sequential search.

3. Comparison of Searching Techniques

Feature	Linear Search	Binary Search	Indexed Sequential Search
Data Arrangement	Unsorted	Sorted	Sorted with index
Method	Sequentially check each element	Divide and search in half	Search using index and then sequentially in a block
Time Complexity (Average Case)	$O(n)$	$O(\log n)$	$O(\log n + m)$ (m = size of block)
Best Use Case	Small datasets or unsorted data	Large sorted datasets	Very large datasets stored in files
Advantage	Simple and easy to implement	Very fast	More efficient for large files
Disadvantage	Slow for large data	Requires sorted data	More complex, needs extra index structure

4. Linear Search (Sequential Search)

Definition

- Linear Search is the **simplest** searching technique. It checks **each element of the list one by one** from the beginning until the key is found or all elements are checked.

Algorithm / Working Steps

- Start from the first element.
- Compare the key with the current element.
- If match is found, return the index.
- If no match is found after checking all elements, return -1.

Example

Search key = 30

Array:

10	20	30	40	50
0	1	2	3	4

Result: 30 is found at index 2.

Pseudocode

```
for i = 0 to n-1
    if A[i] == key
        return i // found
return -1 // not found
```

C Program

```
#include <stdio.h>
int linearSearch(int arr[], int n, int key){
    for(int i = 0; i < n; i++){
        if(arr[i] == key)
            return i;
    }
    return -1;
}
```

Time Complexity

Best Case: $O(1)$
Average Case: $O(n)$
Worst Case: $O(n)$

Advantages

- Simple and easy to implement.
- Works on unsorted data.
- Suitable for small datasets.

Disadvantages

- Slow for large datasets.
- Requires more comparisons.

5. Binary Search

Definition

- Binary Search is an efficient searching technique used **only on sorted data**.
- It repeatedly **divides** the search interval into half and compares the key with the middle element.
- If the key matches the middle element, search is successful. If the key is smaller, search in left half, otherwise in right half.

Algorithm / Working Steps

- Set two pointers: low = 0, high = n - 1.
- Find the middle element mid = (low + high) / 2.
- Compare the key with A[mid].
- If key == A[mid], return mid.
- If key < A[mid], search in left half (high = mid - 1).
- If key > A[mid], search in right half (low = mid + 1).
- Repeat until low > high.

Example

Search key = 50

Sorted Array:

10	20	30	40	50	60	70
			low = 0	mid = 3 (40)		high = 6

50 > 40 → search right half
mid = 5 (60)
50 < 60 → search left half
mid = 4 (50) → Found

Advantages

- Very fast for large datasets.
- Requires fewer comparisons.
- Efficient for sorted data.

Pseudocode

```
low = 0, high = n - 1
while low <= high
    mid = (low + high) / 2
    if A[mid] == key
        return mid
    else if key < A[mid]
        high = mid - 1
    else
        low = mid + 1
return -1
```

C Program

```
#include <stdio.h>
int binarySearch(int arr[], int n, int key){
    int low = 0, high = n - 1;
    while(low <= high){
        int mid = (low + high) / 2;
        if(arr[mid] == key)
            return mid;
        else if(key < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}
```

Time Complexity

Best Case: $O(1)$
Average Case: $O(\log n)$
Worst Case: $O(\log n)$

Disadvantages

- Works only on sorted data.
- Not suitable for unsorted data.

6. Indexed Sequential Search

Definition

- Indexed Sequential Search is a searching technique that combines the advantages of **indexing** and **sequential** search.
- An **index file** is created that contains key values and pointers to data blocks in the main file.
- First, the index is searched to find the appropriate block where the key may be present, and then a sequential search is performed within that block.

Working Steps

- Search the index file to find the largest index key less than or equal to the search key.
- Go to the corresponding block in the main file.
- Perform a sequential search within that block.
- If the key is found, return its position.
- If the key is not found, return -1.

Example

Search key = 70

Index file:		Main file (Data blocks):			
Key (First element)	Block No.	Block B1:	10	20	30
10	B1	Block B2:	40	50	60
40	B2	Block B3:	70	80	90
70	B3				

Search index: largest key ≤ 70 → 70 (B3)
Search sequentially in Block B3.
Result: 70 is found.

Advantages

- More efficient than simple sequential search.
- Suitable for very large files.
- Reduces number of comparisons.
- Works well with data stored in sorted order.

Disadvantages

- Requires extra storage for index file.
- More complex than linear or binary search.
- Performance depends on index structure and block size.

Linear Search



1. Definition

- Linear Search (Sequential Search) is the **simplest** searching technique used to find a given element (key) in a list or array.
- It checks each element **one by one** from the **beginning** until the required element is found or all elements are checked.
- It compares the search key with each element **sequentially**.
- If the key is found, it returns the **position (index)** of the element.
- If the key is **not found** after checking all elements, it returns a failure message (e.g., **-1** or **NULL**).

2. Working Principle

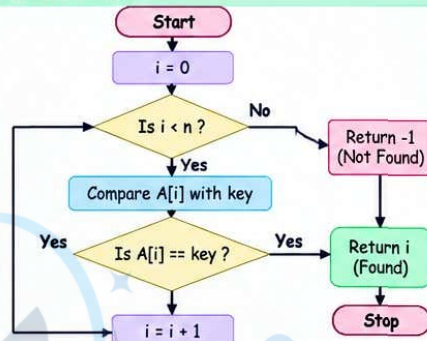
- Start from the first element (index 0).
- Compare the key with the current element.
- If the key matches, return the index/position.
- If the key does not match, move to the next element.
- Repeat the process until the key is found or the list ends.
- If the key is not found, return -1 (or a failure message).

3. Algorithm (Step-by-Step)

Input: Array A[0...n-1], key
Output: Position of key or -1

- Step 1:** Start
Step 2: For $i = 0$ to $n-1$
Step 3: Compare $A[i]$ with key
Step 4: If $A[i] == \text{key}$, return i
Step 5: If not found, move to next element
Step 6: Return -1
Step 7: Stop

4. Flowchart



5. Example - Key Found

Search key = 40

	0	1	2	3	4	5
Array:	10	20	30	40	50	60

- Compare 10 with 40 → not equal.
- Compare 20 with 40 → not equal.
- Compare 30 with 40 → not equal.
- Compare 40 with 40 → equal.

Result: Key 40 is found at index 3.

6. Example - Key Not Found

Search key = 25

	0	1	2	3	4	5
Array:	10	20	30	40	50	60

- Compare 10 with 25 → not equal.
- Compare 20 with 25 → not equal.
- Compare 30 with 25 → not equal.
- Compare 40 with 25 → not equal.
- Compare 50 with 25 → not equal.
- Compare 60 with 25 → not equal.

Result: Key 25 is **not found** (return -1).

7. Pseudocode and C Program

Pseudocode

```

procedure linearSearch(A, n, key)
  for i = 0 to n - 1
    if A[i] == key then
      return i // element found
    end if
  end for
  return -1 // element not found
end procedure
  
```

C Program

```

#include <stdio.h>
int linearSearch(int arr[], int n, int key) {
  for (int i = 0; i < n; i++) {
    if (arr[i] == key)
      return i; // element found
  }
  return -1; // element not found
}

int main() {
  int arr[] = {10, 20, 30, 40, 50, 60};
  int n = sizeof(arr)/sizeof(arr[0]);
  int key = 40;
  int pos = linearSearch(arr, n, key);
  if (pos != -1)
    printf("Key %d found at index %d\n", key, pos);
  else
    printf("Key %d not found\n", key);
  return 0;
}
  
```

8. Time and Space Complexity

Case	Time Complexity	Explanation
Best Case	$O(1)$	Key is found at the first element.
Average Case	$O(n)$	Key may be at any position.
Worst Case	$O(n)$	Key is at the last element or not present.

Space Complexity: $O(1)$ (uses only a few variables)

Where n is the number of elements in the array.



9. Key Points, Advantages and Disadvantages

Key Points

- Linear Search checks each element one by one.
- It works on both sorted and unsorted data.
- Simple to understand and implement.
- Returns the index of the element if found, otherwise -1.
- Time complexity is $O(n)$.

Advantages

- Very simple algorithm.
- Easy to implement.
- Works on both sorted and unsorted data.
- Suitable for small datasets.
- Does not require any special data structure.

Disadvantages

- Time complexity is $O(n)$, which is slow for large data.
- Not suitable for large datasets.
- It compares each element sequentially, so it takes more time.
- Less efficient than Binary Search (for sorted data).



Binary Search



Binary Search works only on **sorted data**.

1. Definition

- Binary Search is an efficient searching technique used to find a **given element (key)** in a **sorted list** or array.
- It repeatedly divides the search space into **half** and compares the key with the **middle element**.
- If the key matches the middle element, the search is successful.
- If the key is smaller, search continues in the **left half**.
- If the key is larger, search continues in the **right half**.
- If the key is not found, it returns a **failure** message (e.g., **-1** or **NULL**).

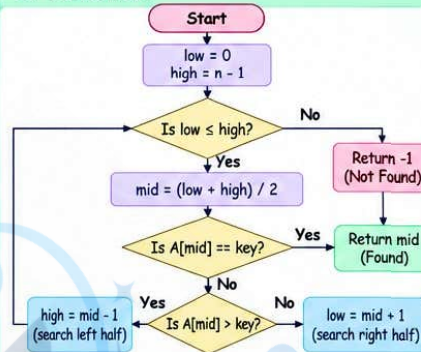
2. Working Principle

- Ensure the data is sorted in ascending order.
- Set two pointers: **low** (start) and **high** (end).
- Calculate the middle index: $\text{mid} = (\text{low} + \text{high}) / 2$
- Compare the middle element with the key.
- If key equals middle element, return the index.
- If key is smaller, search in **left half** ($\text{high} = \text{mid} - 1$). If key is larger, search in **right half** ($\text{low} = \text{mid} + 1$). Repeat until $\text{low} > \text{high}$.

3. Algorithm (Step-by-Step)

- Start
- Set $\text{low} = 0$, $\text{high} = n - 1$
- Repeat steps 4 to 7 while $\text{low} \leq \text{high}$
- Calculate $\text{mid} = (\text{low} + \text{high}) / 2$
- If $A[\text{mid}] == \text{key}$ return mid (**key found**)
- Else if $A[\text{mid}] > \text{key}$ $\text{high} = \text{mid} - 1$ (search left half)
- Else $\text{low} = \text{mid} + 1$ (search right half)
- If $\text{low} > \text{high}$, key is not found return **-1**
- Stop

4. Flowchart



5. Example 1 (Key Found)

Sorted Array:

Index:	0	1	2	3	4	5	6
A[i]:	5	12	18	25	32	40	56

Search key = 25

- $\text{low} = 0$, $\text{high} = 6 \rightarrow \text{mid} = 3 \rightarrow A[3] = 25$
- $25 == 25 \rightarrow$ Key found at index 3.

Result: Key 25 is found at index 3.

6. Example 2 (Key Not Found)

Sorted Array:

Index:	0	1	2	3	4	5	6
A[i]:	5	12	18	25	32	40	56

Search key = 20

- $\text{low} = 0$, $\text{high} = 6 \rightarrow \text{mid} = 3 \rightarrow A[3] = 25$
- $20 < 25 \rightarrow \text{high} = 2$
- $\text{low} = 0$, $\text{high} = 2 \rightarrow \text{mid} = 1 \rightarrow A[1] = 12$
- $20 > 12 \rightarrow \text{low} = 2$
- $\text{low} = 2$, $\text{high} = 2 \rightarrow \text{mid} = 2 \rightarrow A[2] = 18$
- $20 > 18 \rightarrow \text{low} = 3$ (now $\text{low} > \text{high}$)

Result: Key 20 is not found (returns -1).

7. Dry Run Table (Search key = 32)

Step	low	high	mid	A[mid]	Comparison	Action
1	0	6	3	25	$32 > 25$	$\text{low} = 4$
2	4	6	5	40	$32 < 40$	$\text{high} = 4$
3	4	4	4	32	$32 == 32$	Found (index 4)

8. Pseudocode

```

procedure binarySearch(A, n, key)
  low ← 0
  high ← n - 1
  while low ≤ high do
    mid ← (low + high) / 2
    if A[mid] = key then
      return mid
    else if A[mid] > key then
      high ← mid - 1
    else
      low ← mid + 1
  return -1
  
```

9. C Program

```

#include <stdio.h>
int binarySearch(int arr[], int n, int key) {
  int low = 0, high = n - 1, mid;
  while (low <= high) {
    mid = (low + high) / 2;
    if (arr[mid] == key)
      return mid; // key found
    else if (arr[mid] > key)
      high = mid - 1; // search left half
    else
      low = mid + 1; // search right half
  }
  return -1; // key not found
}

int main() {
  int arr[] = {5, 12, 18, 25, 32, 40, 56};
  int n = sizeof(arr) / sizeof(arr[0]);
  int key = 32;
  int pos = binarySearch(arr, n, key);
  if (pos != -1)
    printf("Key %d found at index %d\n", key, pos);
  else
    printf("Key %d not found\n", key);
  return 0;
}
  
```

10. Time and Space Complexity

Case	Time Complexity	Explanation
Best Case	$O(1)$	Key is found at the middle element in the first comparison.
Average Case	$O(\log n)$	Each step reduces the search space to half.
Worst Case	$O(\log n)$	Key is at the extreme end or not present.
Space Complexity	$O(1)$	Uses only a few variables (low, high, mid).

11. Advantages and Disadvantages

Advantages

- ✓ Faster than linear search (time complexity $O(\log n)$).
- ✓ Requires fewer comparisons.
- ✓ Efficient for large datasets.
- ✓ Widely used in real-world applications.

Disadvantages

- ✗ Works only on sorted data.
- ✗ Sorting is required if data is unsorted (extra time).
- ✗ Not suitable for small datasets (where linear search may be simpler).
- ✗ Not efficient for linked lists (due to random access limitation).

12. Key Takeaways

- ★ Binary Search works only on **sorted data**.
- ★ It repeatedly divides the search space into half.
- ★ Time complexity is $O(\log n)$, which is much faster than Linear Search.
- ★ It is most suitable for large, sorted datasets.



Indexed Sequential Search



Indexed Sequential Search combines the advantages of **indexing** and **sequential search**.

1. Definition

- Indexed Sequential Search is a searching technique that uses an **index** to speed up the search process and then performs sequential search within the appropriate block.
- The data file is divided into **multiple blocks** and an index file is created that contains **key values** and **pointers** to the respective blocks.
- First, the index is searched to find the block where the key may be present, and then a **sequential search** is performed within that block.
- It is a **combination of index search and sequential search**.

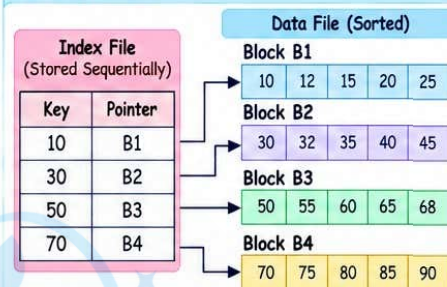
2. Key Features

- Uses an **index file** to reduce the search space.
- Suitable for **large files** stored sequentially on secondary storage (e.g., disk).
- The index contains **key values** and **pointers** to data blocks.
- After locating the correct block using the index, a **sequential search** is performed within that block.
- Requires **sorted data**.
- More efficient** than linear search for large datasets.

3. Working Principle

- Divide the sorted data file into fixed-size blocks.
- Create an **index file** that stores the first (or highest) key value of each block along with a **pointer** to that block.
- Search the **index file** to find the block where the given key may be present.
- Perform a sequential search within that block to find the element.
- If the element is found, return its position. Otherwise, return -1 (not found).

4. Structure



5. Example

Suppose we have the following sorted data file and index file:

Index File		Data File (Sorted)					
Key	Pointer						
10	B1	Block B1					
		10	12	15	20	25	
30	B2	Block B2					
		30	32	35	40	45	
50	B3	Block B3					
		50	55	60	65	68	
70	B4	Block B4					
		70	75	80	85	90	

6. Searching for a Key (Example: Key = 40)

Step 1: Search the index file

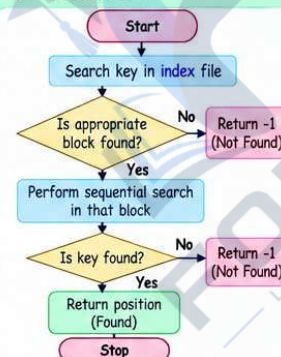
- Find the largest key in the index that is ≤ 40 .
- From the index, $30 \leq 40$, so we go to **Block B2**.

Step 2: Sequential search in Block B2

- Search **40** in Block B2: 30, 32, 35, 40, 45.
- 40** is found at **index 3** (in Block B2).

Result: Key 40 is found in **Block B2** at position 3 (overall position depends on block structure).

7. Flowchart



8. Pseudocode

```

procedure indexedSequentialSearch(key)
  // Step 1: Search in index file
  find the block Bi such that
  Index[i].key ≤ key < Index[i+1].key
  if no such block found
    return -1 // key not found

  // Step 2: Sequential search in block Bi
  for each element x in block Bi
    if x == key
      return position of x // found
  return -1 // key not found
  
```

9. Comparison with Other Searching Techniques

Feature	Linear Search	Binary Search	Indexed Sequential Search
Data Arrangement	Unsorted / Sorted	Sorted	Sorted with index
Method	Sequentially check each element	Divide and search in half	Search index, then sequential search in block
Time Complexity (Average Case)	$O(n)$	$O(\log n)$	$O(\log m + b)$ (m = number of blocks, b = size of block)
Best Use Case	Small datasets	Large sorted datasets	Very large datasets (stored on disk)
Advantage	Simple and easy	Very fast	Efficient for large files
Disadvantage	Slow for large data	Requires sorted data	More complex, needs extra index file

10. Advantages and Disadvantages

Advantages

- ✓ Faster than linear search for large datasets.
- ✓ Suitable for data stored on secondary storage (disk).
- ✓ Combines advantages of indexing and sequential search.
- ✓ Reduces the number of comparisons.
- ✓ Efficient for frequently accessed data.

Disadvantages

- ✗ Requires extra storage for index file.
- ✗ Data must be sorted.
- ✗ More complex to implement than linear search.
- ✗ Less efficient than binary search for small datasets.
- ✗ Index file needs to be maintained when data is updated.

11. Key Takeaways

- ★ Indexed Sequential Search uses an index file to locate the correct block.
- ★ After finding the block, it performs sequential search within that block.
- ★ It is a combination of indexing and sequential search.
- ★ It works only on **sorted data**.
- ★ It is widely used in database systems and file handling for efficient searching in large datasets.



Sorting arranges data in a specific order (ascending or descending).

Sorting Techniques

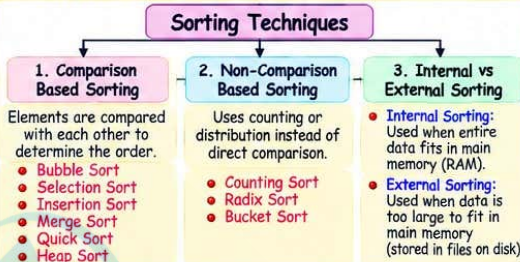


Common Orders:
 • Ascending Order (Small → Large)
 • Descending Order (Large → Small)

1. What is Sorting?

- Sorting is the process of **arranging** elements of a collection in a particular order (**ascending** or **descending**).
- It helps in **searching**, **data analysis** and **efficient data processing**.
- Sorting improves the efficiency of other algorithms like searching.
- Examples: Sorting numbers, names, marks, etc.

2. Types of Sorting Techniques



3. General Steps in Sorting

- Compare elements according to a rule (e.g., < or >).
- If elements are in wrong order, **swap** or **rearrange** them.
- Repeat the process until the entire list is sorted.
- Final output is a **sorted list** in ascending or descending order.

Example

Unsorted Array:

34 12 56 7 23

Sorted (Ascending):

7 12 23 34 56

4. Applications of Sorting

- Makes searching faster (e.g., Binary Search).
- Used in **database systems**, file organization.
- Helpful in **data analysis** and **statistics**.
- Used in graphics, machine learning, and information retrieval.
- Essential for efficient **data management** in real-world applications.

5. Bubble Sort

- Repeatedly compares adjacent elements and swaps them if they are in wrong order.
- Largest element "bubbles" to the end in each pass.
- Simple but **not efficient** for large data.

Example (Ascending Order)

Initial:	5	1	4	2	8
Pass 1:	1	4	2	5	8
Pass 2:	1	2	4	5	8
Pass 3:	1	2	4	5	8

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

6. Selection Sort

- Finds the smallest element from the unsorted part and swaps it with the first element.
- Repeats the process for the remaining elements.
- Simple to implement, but not efficient for large data.

Example (Ascending Order)

Initial:	64	25	12	22	11
Pass 1:	11	25	12	22	64
Pass 2:	11	12	25	22	64
Pass 3:	11	12	22	25	64
Pass 4:	11	12	22	25	64

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

7. Insertion Sort

- Builds a sorted list by inserting each element into its correct position.
- Works like sorting playing cards in hand.
- Efficient for small datasets and nearly sorted data.

Example (Ascending Order)

Initial:	12	11	13	5	6
Pass 1:	11	12	13	5	6
Pass 2:	11	12	13	5	6
Pass 3:	5	11	12	13	6
Pass 4:	5	6	11	12	13

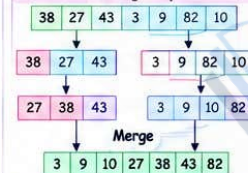
Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

8. Merge Sort

- Divide and conquer algorithm.
- Divide the array into two halves, sort them recursively and merge the sorted halves.
- Stable and efficient for large datasets.

Working Steps



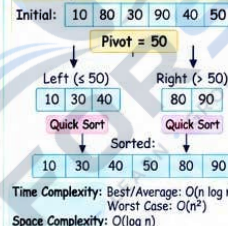
Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

9. Quick Sort

- Divide and conquer algorithm.
- Selects a **pivot** element, partitions the array into smaller parts.
- Recursively sorts the subarrays.
- Very efficient on average, but worst case can be $O(n^2)$.
- In-place sorting (requires less extra memory).

Example (Pivot = last element)



Time Complexity: Best/Average: $O(n \log n)$

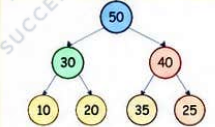
Worst Case: $O(n^2)$

Space Complexity: $O(\log n)$

10. Heap Sort

- Uses a binary heap (usually max-heap) to sort elements.
- Builds a heap from the array.
- Repeatedly extracts the maximum element and places it at the end.
- Efficient and in-place sorting.

Max Heap (Tree Representation)



Sorted Array:

10 20 25 30 35 40 50

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$

11. Comparison of Sorting Techniques

Algorithm	Basic Idea	Best Case	Average Case	Worst Case	Space Complexity	Stable	In-Place
Bubble Sort	Compare adjacent elements	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	Select minimum element	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	Insert elements at correct position	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	Divide, sort and merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	Partition using pivot	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap Sort	Use binary heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Counting Sort	Count and place elements	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes	No
Radix Sort	Sort digit by digit	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Yes	No
Bucket Sort	Distribute into buckets	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	No	No

12. Advantages and Disadvantages

- Advantages**
- Helps in fast searching.
 - Organizes data in a meaningful way.
 - Improves efficiency of many algorithms.
 - Useful in databases, file systems and real-world applications.
 - Different techniques available for different types of data.

- Disadvantages**
- Some sorting algorithms are time-consuming (e.g., $O(n^2)$).
 - May require extra memory (e.g., Merge Sort, Counting Sort).
 - Not all algorithms are stable.
 - Performance varies based on input size and data type.
 - Complex to implement for large datasets.

13. Key Takeaways

- Sorting arranges data in a specific order (ascending or descending).
- There are comparison-based and non-comparison-based techniques.
- Choice of algorithm depends on data size, memory, and application.
- Merge Sort and Quick Sort are efficient for large datasets.
- Counting Sort, Radix Sort and Bucket Sort are useful for special cases (integers).
- Each algorithm has its own advantages and limitations.



Bubble Sort



Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

1. Definition

- Bubble Sort is a simple **comparison-based** sorting algorithm.
- It repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.
- After each pass, the largest element "**bubbles**" to the end of the array.
- It continues this process for $(n - 1)$ passes or until the array is sorted.
- It is a **stable** and **in-place** sorting algorithm, but not efficient for large datasets.

2. Working Principle

- Compare the first two adjacent elements.
- If the first element is greater than the second, swap them.
- Move to the next pair of adjacent elements and repeat the step.
- Continue this process till the end of the array. The largest element will be at the last position.
- Repeat the above process for the remaining $(n - 1)$ elements.
- Stop when no swaps are required (i.e., the array is already sorted).

3. Example (Ascending Order)

Initial Array : 5 1 4 2 8

Pass 1 :
 1 5 4 2 8 ($5 > 1$) Swap
 1 4 5 2 8 ($5 > 4$) Swap
 1 4 2 5 8 ($5 > 2$) Swap
 1 4 2 5 8 ($5 < 8$) No Swap

Largest element 8 is at last position.

Pass 2 :
 1 4 2 5 8 ($4 > 2$) Swap
 1 2 4 5 8 ($4 < 5$) No Swap

Pass 3 : 1 2 4 5 8 No Swap (Array is sorted)

6. Example (Descending Order)

Initial Array : 3 6 2 8 1

Pass 1 :
 6 3 8 2 1 ($3 < 6$) Swap
 6 8 3 2 1 ($3 < 8$) Swap
 6 8 3 2 1 ($3 > 2$) No Swap
 6 8 3 2 1 ($2 > 1$) No Swap

Largest element 8 is at last position (for descending, we place smallest at end).

Pass 2 :
 8 6 3 2 1 ($6 < 8$) Swap
 8 6 3 2 1 ($6 > 3$) No Swap
 8 6 3 2 1 ($3 > 2$) No Swap

Pass 3 : 8 6 3 2 1 (No Swap)

Sorted Array (Descending): 8 6 3 2 1

7. C Program

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {5, 1, 4, 2, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array (Ascending): ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

8. Time and Space Complexity

Case	Time Complexity	Explanation
Best Case	$O(n)$	Array is already sorted (no swaps).
Average Case	$O(n^2)$	Comparisons and swaps are needed.
Worst Case	$O(n^2)$	Array is sorted in reverse order.

Space Complexity : $O(1)$
(In-place sorting, no extra memory required)

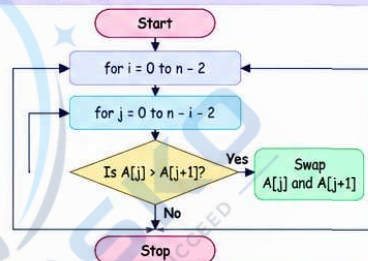
9. Characteristics

- Simple and easy to understand.
- Stable sorting algorithm (maintains relative order of equal elements).
- In-place sorting (uses constant extra space).
- Does not require extra data structures.
- Inefficient for large datasets.
- Mainly used for small datasets or educational purposes.

4. Pseudocode

```
procedure bubbleSort(A, n)
    for i = 0 to n - 2
        for j = 0 to n - i - 2
            if A[j] > A[j + 1] then
                swap(A[j], A[j + 1])
            end if
        end for
    end for
end procedure
```

5. Flowchart



10. Advantages and Disadvantages

Advantages	Disadvantages
✓ Easy to implement.	✗ Time complexity is $O(n^2)$ (slow for large datasets).
✓ Simple and intuitive.	✗ Not suitable for large datasets.
✓ Stable sorting algorithm.	✗ Performs many unnecessary comparisons and swaps.
✓ In-place sorting ($O(1)$ space).	✗ Less efficient compared to other sorting algorithms (Quick Sort, Merge Sort).
✓ Works well for small datasets.	

11. Key Takeaways

- ★ Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.
- ★ After each pass, the largest element moves to its correct position.
- ★ Time complexity is $O(n^2)$.
- ★ It is a stable and in-place sorting algorithm.
- ★ Useful for small datasets and easy to understand.



Selection Sort repeatedly finds the minimum element and places it at the correct position.

Selection Sort



In each pass, the smallest element from the unsorted part is selected and swapped with the first element of that part.

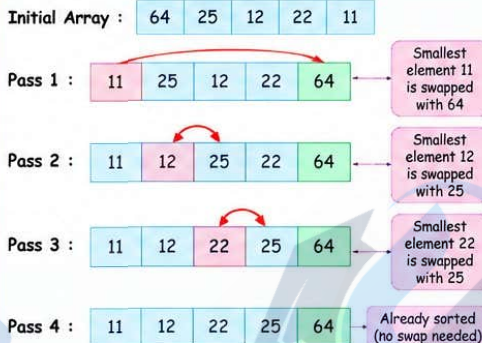
1. Definition

- Selection Sort is a simple **comparison-based** sorting algorithm.
- It repeatedly finds the **smallest element** from the unsorted part of the array and places it at the beginning.
- The array is divided into two parts: a **sorted part** and an **unsorted part**.
- It performs $(n - 1)$ passes to sort an array of n elements.
- It is an **in-place** sorting algorithm but **not stable** by default.

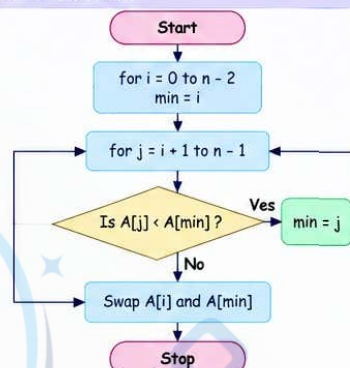
2. Working Principle

- Start from the first element and assume it is the minimum.
- Compare it with all elements in the unsorted part.
- If a smaller element is found, update the index of minimum element.
- After comparing all elements, swap the minimum element with the first element.
- Move to the next position and repeat the process for the remaining elements.
- Continue until the entire array is sorted.

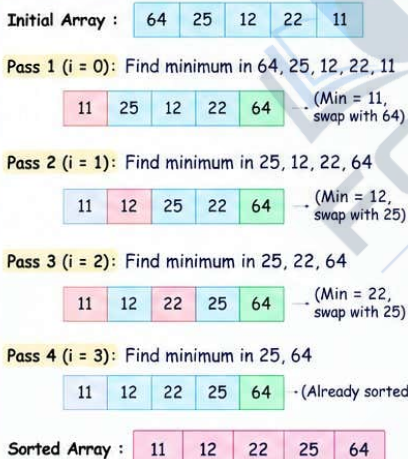
3. Example (Ascending Order)



4. Flowchart



5. Example in Detail



6. Pseudocode

```

procedure selectionSort(A, n)
  for i = 0 to n - 2
    min = i
    for j = i + 1 to n - 1
      if A[j] < A[min] then
        min = j
    end for
    if min != i then
      swap(A[i], A[min])
    end if
  end for
end procedure
  
```

7. C Program

```

#include <stdio.h>

void selectionSort(int arr[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int min = i;
    for (int j = i + 1; j < n; j++) {
      if (arr[j] < arr[min]) {
        min = j;
      }
    }
    if (min != i) {
      int temp = arr[i];
      arr[i] = arr[min];
      arr[min] = temp;
    }
  }
}

int main() {
  int arr[] = {64, 25, 12, 22, 11};
  int n = sizeof(arr) / sizeof(arr[0]);
  selectionSort(arr, n);
  printf("Sorted array (Ascending): ");
  for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
  return 0;
}
  
```

8. Time and Space Complexity

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity : $O(1)$
(In-place sorting, no extra memory required)

9. Characteristics

- Simple and easy to implement.
- In-place sorting algorithm.
- Not stable (by default).
- Performs exactly $(n - 1)$ passes.
- Number of comparisons: $n(n - 1) / 2$.
- Suitable for small datasets.
- Not efficient for large datasets.

10. Advantages and Disadvantages

Advantages

- Simple and easy to understand.
- In-place sorting ($O(1)$ space).
- Consistent performance ($O(n^2)$ in all cases).
- Useful for small datasets.

Disadvantages

- Time complexity is $O(n^2)$ (slow for large datasets).
- Not stable (by default).
- Performs more comparisons than some other algorithms (like Merge Sort).
- Not suitable for large datasets.

11. Key Takeaways

- Selection Sort selects the smallest element and places it at the correct position.
- It divides the array into sorted and unsorted parts.
- It is an in-place sorting algorithm.
- Time complexity is $O(n^2)$.
- It is easy to implement and understand.
- Useful for small datasets and educational purposes.



Insertion Sort builds a sorted array one element at a time by inserting each element into its correct position.

Insertion Sort



In each step, the current element is compared with the elements in the sorted part and shifted to its correct position.

1. Definition

- Insertion Sort is a simple **comparison-based** sorting algorithm.
- It builds the final sorted array **one element at a time** by inserting each element into its correct position.
- The array is divided into two parts:
 - Sorted part** (left side)
 - Unsorted part** (right side)
- At each step, an element from the unsorted part is taken and inserted into its correct position in the sorted part.
- It is an **in-place** sorting algorithm and **stable** by default.

2. Working Principle

- Start with the second element (from index 1) and consider the first element as a sorted part.
- Pick the current element (key) from the unsorted part.
- Compare the key with elements in the sorted part (from right to left).
- Shift elements that are greater than the key to one position ahead.
- Insert the key at its correct position.
- Repeat the process for all elements until the entire array is sorted.

3. Example (Ascending Order)

Initial Array : 12 11 13 5 6

Pass 1
(i = 1)
Key = 11

12 11 13 5 6

11 < 12
Shift 12 right
and insert 11

Pass 2
(i = 2)
Key = 13

11 12 13 5 6

13 > 12
No shifting
(required)

Pass 3
(i = 3)
Key = 5

11 12 13 5 6
11 12 13 13 6
11 12 12 13 6
11 5 12 13 6

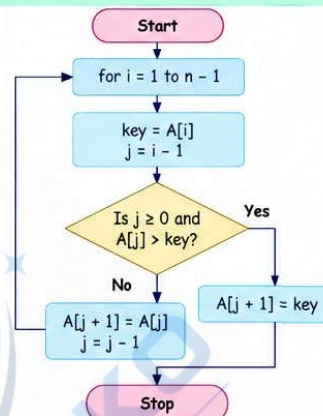
5 < 13 → shift
5 < 12 → shift
5 < 11 → shift
Insert 5

Pass 4
(i = 4)
Key = 6

11 5 12 13 6
5 6 11 12 13

6 < 13 → shift
6 < 12 → shift
6 < 11 → shift
Insert 6

4. Flowchart



5. Example in Detail

Initial Array : 12 11 13 5 6

Pass 1: Insert 11 into sorted part

12 11 13 5 6 → 11 12 13 5 6

Pass 2: Insert 13 into sorted part

11 12 13 5 6 → 11 12 13 5 6

Pass 3: Insert 5 into sorted part

11 12 13 5 6 → 11 5 12 13 6

Pass 4: Insert 6 into sorted part

11 5 12 13 6 → 5 6 11 12 13

Final Sorted Array : 5 6 11 12 13

6. Pseudocode

```

procedure insertionSort(A, n)
  for i = 1 to n - 1
    key = A[i]
    j = i - 1
    while j ≥ 0 and A[j] > key do
      A[j + 1] = A[j]
      j = j - 1
    end while
    A[j + 1] = key
  end for
end procedure
    
```

Note: The first element is always in the sorted part, so we start from index 1.

7. C Program

```

#include <stdio.h>

void insertionSort(int arr[], int n) {
  for (int i = 1; i < n; i++) {
    int key = arr[i];
    int j = i - 1;
    while (j >= 0 && arr[j] > key) {
      arr[j + 1] = arr[j];
      j--;
    }
    arr[j + 1] = key;
  }
}

int main() {
  int arr[] = {12, 11, 13, 5, 6};
  int n = sizeof(arr) / sizeof(arr[0]);
  insertionSort(arr, n);
  printf("Sorted array: ");
  for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
  return 0;
}
    
```

8. Time and Space Complexity

Case	Time Complexity
Best Case	$O(n)$ (Already sorted array)
Average Case	$O(n^2)$
Worst Case	$O(n^2)$ (Reverse sorted array)
Space Complexity : $O(1)$ (In-place sorting, no extra memory required)	

9. Characteristics

- Simple and easy to implement.
- In-place sorting algorithm.
- Stable sorting algorithm (maintains relative order of equal elements).
- Efficient for small datasets.
- Adaptive (performs well when array is partially sorted).
- Not efficient for large datasets.

10. Advantages and Disadvantages

Advantages

- Simple to understand and implement.
- In-place sorting ($O(1)$ space).
- Stable algorithm.
- Efficient for small and nearly sorted datasets.
- Works well in online algorithms (data arrives sequentially).

Disadvantages

- Time complexity is $O(n^2)$ (slow for large datasets).
- Not suitable for large datasets.
- Requires many comparisons and shifts.

11. Key Takeaways

- Builds a sorted array by inserting each element.
- Compares and shifts elements from right to left.
- Stable and in-place algorithm.
- Time complexity: $O(n^2)$ (except best case $O(n)$).
- Useful for small and partially sorted datasets.